Project Documentation Group 2

Andrei Ursachi, Miguel Ángel Simonet Sánchez,
and Salmanul Faris Kodungookaran Abdul Muthaleeb

University of Twente
a.ursachi@student.utwente.nl, m.a.simonetsanchez@student.utwente.nl,
s.f.kodungookaranabdulmuthaleeb@student.utwente.nl

April 14, 2026

According to the following documentation of computational results for the graph isomorphism project, we aim at a group-average grade of 10, which is in accordance with the grading rules. The work has been distributed equally among the three group members, as evidenced by Table 11.

Team member	grade (deviation from average)
Andrei Ursachi (s3351912)	10.0 (+0.0)
Miguel Ángel Simonet Sánchez (s3398723)	10.0 (+0.0)
Salmanul Faris K. Abdul Muthaleeb (s3401731)	10.0 (+0.0)

Table 1: *Suggested grades*

On computational results. All computations were performed on the same machine (Linux, AMD64). Each measurement was run 3 times; the median is reported. All times are wall-clock seconds. Our implementation uses only the Python 3.10+ standard library with no external dependencies.

1. Category 6: Basic Problem Instances

All basic problem instances have been solved correctly during the programming competition. Table 2 shows the computation times on the mandatory colour-refinement sample instances. Every instance is solved correctly and efficiently.

Architecture overview. Our solver is invoked via `python3 branching.py <file>`. It automatically detects the problem type from the filename (`*GI.grl` for GI, `*Aut.grl/gr` for #Aut, `*GIAut.grl` for both). The pipeline is:

1. **Preprocessing**: detect trees/forests (solved in $O(n)$ via AHU).
2. **Colour refinement**: compute a stable colouring using the fast $O(m \log n)$ Hopcroft-style algorithm.
3. **Branching**: if the colouring is balanced but not discrete, individualise-and-refine.
4. **Automorphism counting**: use generating sets with Lemma 5.11 pruning and the orbit-stabiliser theorem.

instance	correctly solved	comp. time (s)
colorref_smallexample_2_49	✓	0.001
colorref_smallexample_4_7	✓	<0.001
colorref_smallexample_4_16	✓	<0.001
colorref_smallexample_6_15	✓	0.001
colorref_largeexample_4_1026	✓	0.080
colorref_largeexample_6_960	✓	0.036
test_empty	✓	<0.001
test_cycles	✓	0.002
test_trees	✓	<0.001
test_3reg	✓	0.002
test_stab	✓	<0.001
test_iter	✓	0.001
test_cref9	✓	<0.001
cref9vert3comp_10_27	✓	0.002

Table 2: *Computation times basic instances.*

Basic colour refinement. Algorithm 1 shows the basic colour refinement procedure (Algorithm 1 from the reader, Chapter 2). It iteratively refines vertex colours based on the sorted multiset of neighbour colours until stability.

Algorithm 1 Basic Colour Refinement

Require: A set of graphs $\mathcal{G} = \{G_1, \ldots, G_\ell\}$
Ensure: A stable colouring α of $\mathcal{G}$

```
for all v ∈ V(G) do
    α(v) ← 1                                          ▷ uniform initial colouring
end for
repeat
    changed ← False
    for each vertex v ∈ V(G) do
        sig(v) ← (α(v), sorted(α(u) : u ∈ N(v)))
    end for
    Partition vertices by signature; assign new colours 0, 1, 2, ...
    if number of distinct colours increased then
        changed ← True
    end if
until not changed
return α
```

The basic algorithm has time complexity $O(n^2 m)$ per iteration (where $n = |V|$ and $m = |E|$), with at most n iterations, giving $O(n^3 m)$ worst-case. Our implementation is in `colorref.py` (see Appendix C).

2. Category +1: Implementation of Fast Partition Refinement

We have implemented the fast partition refinement algorithm based on Hopcroft's algorithm for DFA minimisation of finite automata [1], adapted for graph colour refinement as described in Chapter 4 of the reader.

Algorithm. Instead of iterating over all colour classes each round ($O(n^2m)$ for basic refinement), we maintain a work-queue of colour classes. When refining with class C, each other class D is split by the number of neighbours each vertex in D has inside C. When a class splits, only the *smaller* fragments are added to the queue (Lemma 4.9 from the reader). This ensures each vertex enters the queue at most $O(\log n)$ times, yielding $O(m \log n)$ total.

Data structures. Colour classes are stored as Python `set` objects for $O(1)$ membership testing and $O(|C|)$ iteration. A dictionary maps each vertex to its current colour for $O(1)$ lookup. The queue is a `collections.deque`. The `Partition` class (see `fast_colorref.py`, Appendix D) encapsulates this state. For performance, inner loops are "inlined" with local variable aliases to avoid repeated dictionary lookups.

Algorithm 2 Fast Colour Refinement (Hopcroft-style)

Require: Vertices V, adjacency *adj*, initial colouring α_0
Ensure: Stable partition Π

```
Build initial partition Π from α_0; add all classes to queue Q
while Q ≠ ∅ do
    C ← Q.popleft()
    for each u ∈ C, each neighbour w of u do
        count[w] ← count[w] + 1
    end for
    Group affected vertices by their current colour class D
    for each affected class D do
        Subpartition D by the value of count[·]
        if D splits into fragments then
            Keep the largest fragment in D's colour
            Assign new colours to the other fragments
            if D ∈ Q or fragment is new then
                Add each non-largest fragment to Q
            end if
        end if
    end for
    Reset count
end while
return Π
```

Results. Table 3 shows the computation times for the mandatory sample instances with both basic and fast partition refinement. The fast algorithm is consistently faster and exhibits near-linear scaling, while the basic algorithm scales quadratically.

instance	instance size $\lvert V \rvert$	# graphs	comp. times (s) standard	fast part. ref.
colorref_smallexample_2_49	98	2	0.001	<0.001
colorref_smallexample_4_7	28	4	<0.001	<0.001
colorref_smallexample_4_16	64	4	<0.001	<0.001
colorref_smallexample_6_15	90	6	0.001	<0.001
colorref_largeexample_4_1026	4104	4	0.066	0.022
colorref_largeexample_6_960	5760	6	1.461	0.008

Table 3: *Computation times to the coarsest stable colouring with basic and fast partition refinement.*

The speedup is most dramatic on `colorref_largeexample_6_960` (6×960 vertices), where fast refinement is **182**× faster than basic refinement (0.008 s vs. 1.461 s). On `colorref_largeexample_4_1026` (4×1026 vertices), the speedup is 3× (0.022 s vs. 0.066 s). The smaller instances are too fast for meaningful comparison.

Linear running time on `threepaths` instances. Table 4 shows the running times on the `threepaths` benchmark instances (graphs consisting of 3 disjoint paths). These instances are specifically designed to expose the quadratic behaviour of basic colour refinement on path-like structures. The fast algorithm shows **linear scaling**: doubling the instance size approximately doubles the running time (ratio $\approx 2\times$). The basic algorithm shows quadratic scaling and becomes infeasible beyond 2 560 vertices, while the fast algorithm handles 10 240 vertices in 0.015 s.

instance	$\lvert V \rvert$	comp. times (s) basic	fast	speedup
threepaths80	80	0.001	<0.001	—
threepaths160	160	0.004	<0.001	16×
threepaths320	320	0.017	0.001	33×
threepaths640	640	0.065	0.001	66×
threepaths1280	1 280	0.260	0.002	140×
threepaths2560	2 560	0.952	0.004	223×
threepaths5120	5 120	—	0.007	—
threepaths10240	10 240	—	0.015	—

Table 4: *Running times on `threepaths` instances. "—" means > 600 s. The fast algorithm scales linearly; the basic algorithm scales quadratically.*

The implementation is in `fast_colorref.py` (Appendix D). The `threepaths` instances are generated by `generate_threepaths.py`.

3. Category +1: Using Generating Sets for Computing $\lvert \mathrm{Aut}(G) \rvert$

We implemented the automorphism group computation from Chapter 5 of the reader, using generating sets and the orbit-stabiliser theorem instead of enumerating all automorphisms.

Algorithm. We compare graph G to a copy H and build a generating set X of $\mathrm{Aut}(G)$ via Algorithm 9 (UpdateGeneratingSet). The key optimisation is **Lemma 5.11 pruning**: after exploring

the trivial branch (mapping $x \to x'$, which collects all automorphisms fixing x), each non-trivial branch ($x \to y$ for $y \neq x'$) is explored only until a single automorphism is found, then pruned. This is correct because, by Lemma 5.11, one generator combined with the stabiliser already generates all automorphisms of that type.

The group order $|\langle X \rangle|$ is computed via the **orbit-stabiliser theorem** (Proposition 5.18): $|H| = |H_\alpha| \cdot |\alpha^H|$, applied recursively through a Schreier-Sims stabiliser chain. Orbits and transversals are computed via BFS (Algorithm 10); stabiliser generators via Schreier's Lemma.

Algorithm 3 UpdateGeneratingSet (Algorithm 9 with Lemma 5.11 pruning)

Require: Graphs G, H (where H is a copy of G), sequences D, I
Ensure: Generating set X of $\mathrm{Aut}(G)$

```
X ← ∅
procedure UPDATE(D, I)
    Π ← FASTCOLOURREFINE(G + H, α_{D,I})
    if Π is unbalanced then return
    end if
    if Π defines a bijection then
        Extract permutation σ; if σ ≠ id, add σ to X
        return
    end if
    Choose smallest non-trivial colour class C_G, C_H
    x ← C_G[0];   x' ← copy of x in H
                                    ▷ Trivial branch: x → x' (collects stabiliser generators)
    D.append(x);   I.append(x')
    UPDATE(D, I)
    I.pop()
                                    ▷ Non-trivial branches: x → y for y ∈ C_H \ {x'}
    for each y ∈ C_H, y ≠ x' do
        I.append(y)
        FINDONE                     ▷ find one automorphism, then prune
        I.pop()
    end for
    D.pop()
end procedure
```

Algorithm 4 Orbit and Transversal (Algorithm 10)

Require: Generators X, element α
Ensure: Orbit Δ, transversal T

```
Δ ← {α};   T[α] ← id;   Q ← [α]
while Q ≠ ∅ do
    β ← Q.popleft()
    for each g ∈ X do
        γ ← g(β)
        if γ ∉ Δ then
            Δ ← Δ ∪ {γ};   T[γ] ← g ∘ T[β]
            Q.append(γ)
        end if
    end for
end while
return (Δ, T)
```

Algorithm 5 Group Order via Stabiliser Chain (Proposition 5.18)

Require: Generators X, elements Ω
Ensure: $|\langle X \rangle|$

```
Choose base B = (β_1, ..., β_k) from moved elements
current_gens ← X
order ← 1
for each β_i ∈ B do
    (Δ_i, T_i) ← OrbitAndTransversal(current_gens, β_i)
    order ← order × |Δ_i|
    Compute Schreier generators for Stab(β_i):
    for each γ ∈ Δ_i, each g ∈ current_gens do
        h ← T_i[g(γ)]^{-1} ∘ g ∘ T_i[γ]
        if h ≠ id then
            Add h to next-level generators
        end if
    end for
    current_gens ← next-level generators
end for
return order
```

The `Permutation` class (`permutation.py`, Appendix F) stores only non-fixed-point mappings for efficiency. Composition follows the reader's convention: $(p \cdot q)(x) = p(q(x))$.

Results. Table 5 compares full enumeration (Algorithm 2) against the generating-set approach (Algorithm 9 + orbit-stabiliser). The improvement is dramatic: graphs with millions of automorphisms are solved in milliseconds instead of being infeasible.

instance	$\lvert V\rvert$	$\lvert\mathrm{Aut}(G)\rvert$	enum. (s)	genset (s)
trees11	11	6	0.001	<0.001
cubes3	8	48	0.002	0.001
cubes4	16	384	0.031	0.007
cubes5	32	3 840	0.776	0.051
cubes6	64	46 080	—	0.456
cubes7	128	645 120	—	9.7
torus24	24	96	0.013	0.013
wheeljoin14	15	1 600	0.125	0.006
regulartwins	90	16 777 216	—	0.123
wheelstar12	30	1 935 360	—	0.411
bigtrees1	44	442 368	95.06	<0.001
bigtrees3	227	$\approx 2.8 \times 10^{33}$	—	<0.001

Table 5: *Full enumeration vs. generating-set approach for computing $|Aut(G)|$. "—" means $> 600\,s$.*

The implementation is in `branching.py` (Appendix E) and `permutation.py` (Appendix F).

4. Category $+X$: Additional Techniques for Faster Algorithms

4.1. Branching Rules

We use the branching rule of always selecting the *smallest* non-trivial colour class (with ≥ 2 vertices per graph) to branch on. This minimises the branching factor at each level of the recursion tree, reducing the total number of recursive calls. The alternative (e.g., choosing the largest class or an arbitrary class) was tested and found to be consistently slower across all instances.

Algorithm 6 CountIsomorphisms with Smallest-Class Branching (Algorithm 2)

Require: Graphs G, H, sequences D, I
Ensure: Number of isomorphisms from G to H

```
function CountIso(D, I)
    Π ← FastColourRefine(G + H, α_{D,I})
    if Π is unbalanced then return 0
    end if
    if Π defines a bijection then return 1
    end if
    (C_G, C_H) ← smallest non-trivial colour class in both graphs
    x ← C_G[0]
    count ← 0
    for each y ∈ C_H do
        D.append(x);   I.append(y)
        count ← count + CountIso(D, I)
        D.pop();   I.pop()
    end for
    return count
end function
```

instance	$\lvert V\rvert$	smallest class (s)	arbitrary class (s)
cubes5 GI	32	0.009	0.015
cubes6 GI	64	0.038	0.091
torus72 GI	72	0.201	0.347
modulesD GI	48	2.60	4.81

Table 6: *Effect of branching rule on GI computation times.*

4.2. Preprocessing

4.2.1 Trees (grade $+1$)

We have implemented a polynomial-time algorithm for GI and #Aut on trees and forests, based on the AHU algorithm (Section 6.2 of the reader). Our algorithm handles both trees *and* forests (including the permutation factor for isomorphic components).

Detection. A graph is a tree iff it is connected and has $|E| = |V| - 1$. A forest is detected by checking $|E| = |V| - |\text{components}|$. Connected components are found via BFS (Algorithm 7).

Algorithm 7 Find Connected Components (BFS)

Require: Graph G
Ensure: List of components (each a list of vertices)

```
visited ← ∅;   components ← []
for each v ∈ V(G) do
    if v ∉ visited then
        BFS from v: collect all reachable vertices into C
        components.append(C)
        visited ← visited ∪ C
    end if
end for
return components
```

Canonical labelling (AHU). We find the center of the tree (1 or 2 vertices) by iteratively removing leaves (Algorithm 8). The AHU label is computed recursively: each vertex's label is the sorted tuple of its children's labels, with leaves labelled $()$ (Algorithm 9). For two centers, we take the lexicographic minimum.

Algorithm 8 Find Tree Center

Require: Tree vertices V
Ensure: Center vertices (1 or 2)

```
Compute degree of each vertex within the tree
remaining ← V;   leaves ← {v : deg(v) ≤ 1}
while |remaining| > 2 do
    new_leaves ← ∅
    for each v ∈ leaves do
        Remove v from remaining
        for each neighbour u of v in remaining do
            deg(u) ← deg(u) − 1
            if deg(u) = 1 then add u to new_leaves
            end if
        end for
    end for
    leaves ← new_leaves
end while
return remaining
```

Algorithm 9 AHU Canonical Label

Require: Root r, parent p, vertex set S
Ensure: Canonical label (nested tuple)

```
children_labels ← []
for each child c of r (i.e., c ∈ N(r) ∩ S, c ≠ p) do
    children_labels.append(AHULabel(c, r, S))
end for
return sorted_tuple(children_labels)
```

GI. Two trees are isomorphic iff their canonical labels are equal.

#Aut. For a rooted tree, $|\mathrm{Aut}(T)| = \prod_v \big(k_v! \cdot \prod_{\text{children } c} |\mathrm{Aut}(T_c)|\big)$ where k_v is the multiplicity of identical subtree labels. For two centers with isomorphic halves, multiply by 2. For forests, multiply by $k!$ per group of k isomorphic components.

Algorithm 10 Tree Automorphism Count

Require: Tree T (as vertex list)
Ensure: $|\mathrm{Aut}(T)|$

```
centers ← FindCenter(T)
if |centers| = 1 then
    (aut, _) ← CountRooted(centers[0], None)
    return aut
else
    (aut_a, label_a) ← CountRooted(centers[0], centers[1])
    (aut_b, label_b) ← CountRooted(centers[1], centers[0])
    total ← aut_a × aut_b
    if label_a = label_b then
        total ← total × 2                          ▷ swap isomorphic halves
    end if
    return total
end if
```

Results. Table 7 shows the dramatic speedup. The AHU algorithm solves all tree instances in microseconds, while even the generating-set approach takes seconds on large trees when forced to use branching instead.

instance	$\lvert V\rvert$	$\lvert\mathrm{Aut}(G)\rvert$	branching (s)	AHU (s)
trees11	11	6	0.001	<0.001
trees36	36	2	<0.001	<0.001
trees90	90	6 912	<0.001	<0.001
bigtrees1	44	442 368	95.06*	<0.001
bigtrees2	71	$\approx 8.0 \times 10^{13}$	—*	<0.001
bigtrees3	227	$\approx 2.8 \times 10^{33}$	—*	<0.001

Table 7: *Branching vs. AHU for trees and forests. *Full enumeration without generating sets. "—" means* $> 600\,s$.

The implementation is in `preprocessing.py` (Appendix G).

4.2.2 Preprocessing using Twins

We implemented both detection and iterative elimination of true twins and false twins, as described in Section 6.3 of the reader.

Detection. True twins (adjacent, $N(u) \setminus \{v\} = N(v) \setminus \{u\}$) and false twins ($N(u) = N(v)$, not adjacent) are detected in $O(n + m)$ time by hashing neighbourhood signatures.

Algorithm 11 Find Twin Groups

Require: Graph G
Ensure: Lists of true twin groups and false twin groups
1: **for** each vertex $v \in V(G)$ **do**
2: *false_sig*$[v] \leftarrow$ frozenset$(N(v))$
3: *true_sig*$[v] \leftarrow$ frozenset$(N(v) \cup \{v\})$
4: **end for**
5: Group vertices by *false_sig*; keep groups of size ≥ 2 as *false twins*
6: Group vertices by *true_sig*; keep groups of size ≥ 2 as *true twins*
7: **return** (*true twin groups*, *false twin groups*)

Iterative reduction. Our `reduce_twins()` function iteratively detects and collapses twin groups: for each group of k (false) twins, one representative is kept and the others removed, contributing a factor of $k!$ to the automorphism count. After each round, the reduced graph may expose new twin groups, so the process repeats until stable. The reduced graph receives an initial colouring that encodes the twin history of each representative.

Integration. The `reduce_twins()` function is integrated into the #Aut pipeline in `count_automorphisms()`: before branching, twins are iteratively collapsed, the automorphism factor $k!$ per group is accumulated, and the reduced graph (with a pre-colouring encoding twin structure) is passed to `_count_aut_core()`. The pre-colouring assigns each representative a colour derived from the twin type (true/false), group size, and elimination round, so that colour refinement on the reduced graph can distinguish structurally different representatives. If no twins are found (reduced graph has the same size), the reduction is skipped.

Table 8 shows the effect of twin reduction on twin-heavy instances: the twin-reduced pipeline solves these graphs faster by shrinking the graph before branching.

instance	$\lvert V\rvert$	$\lvert\mathrm{Aut}(G)\rvert$	without twins (s)	with twins (s)
regulartwins g0	90	16 777 216	0.12	0.09
regulartwins g1	90	16 777 216	0.12	0.09
cographs1 g0	22	5 971 968	0.04	0.01
cographs1 g1	22	995 328	0.01	<0.01

Table 8: *Computation times for twin-heavy instances with and without twin reduction.*

4.3. Connected Component Decomposition

For disconnected graphs, we decompose into connected components via BFS (Algorithm 7) before processing. This applies to *all* disconnected graphs, not just forests: each component's automorphisms are computed independently (using the appropriate algorithm—AHU for trees, generating sets for general graphs), and isomorphic components contribute an additional $k!$ factor for each group of k isomorphic components. Component isomorphism is tested by first comparing vertex and edge counts, then using the branching algorithm for exact comparison. The decomposition runs in $O(n + m)$ and adds negligible overhead.

5. Category +1: Additional and Genuinely New Ideas

5.1. Unified Solver with Automatic Problem Detection

We designed a unified solver architecture (`branching.py`, function `solve()`) that automatically detects the problem type from the filename and selects the optimal algorithm pipeline. This architecture composes all implemented techniques into a single entry point:

1. **Filename parsing**: `*GI.grl` → GI only; `*Aut.grl/gr` → #Aut only; `*GIAut.grl` → both.
2. **Preprocessing dispatch**: if all graphs are trees, route to the $O(n)$ AHU pipeline for both GI (canonical label comparison) and #Aut (factorial formula). Otherwise, use the general branching pipeline.
3. **GI fast-path**: colour refinement first separates obviously non-isomorphic graphs by their colour signature (no branching needed). Branching is applied only within groups of graphs that colour refinement cannot distinguish.
4. **#Aut fast-path**: forests are detected and solved via AHU. For general graphs, the generating-set approach with Lemma 5.11 pruning is used.

Algorithm 12 FindEquivalenceClasses (GI with colour refinement fast-path)

Require: List of graphs $G_0, \ldots, G_{\ell-1}$, adjacency *adj*
Ensure: Equivalence classes (lists of graph indices)

```
Π ← FastColourRefine(⋃_i G_i)
Group graphs by colour signature sig(G_i) = sorted((colour, count))
for each group with |group| > 1 do
    Pairwise test isomorphism via CountIso (Algorithm 6)
    Partition group into equivalence classes
end for
return all equivalence classes
```

This design avoids unnecessary branching on most instances: colour refinement alone resolves the vast majority of graph pairs, and branching is invoked only when needed. Combined with the preprocessing pipeline, this produces a solver that handles all test instances correctly and efficiently.

instance	$\lvert V\rvert$	time (s)
colorref_smallexample_2_49	98	0.001
colorref_smallexample_4_7	28	<0.001
colorref_smallexample_4_16	64	<0.001
colorref_smallexample_6_15	90	0.001
colorref_largeexample_4_1026	4104	0.080
colorref_largeexample_6_960	5760	0.036
test_cycles	varies	0.002
test_3reg	varies	0.002
cref9vert3comp_10_27	270	0.002

Table 9: *End-to-end solver times on all sample instances.*

6. Summary of Implemented Features

feature	implemented	grade contribution
Basic colour refinement + branching (GI & #Aut)	✓	Category 6 (pass)
Fast partition refinement (Hopcroft-style, $O(m \log n)$)	✓	+1
Generating sets + orbit-stabiliser for #Aut	✓	+1
Branching rule (smallest colour class)	✓	$+\frac{1}{2}$
Tree/forest preprocessing (AHU, $O(n)$)	✓	+1
Twins detection & iterative reduction (integrated)	✓	+1
Component decomposition (all disconnected graphs)	✓	—
Unified solver with auto-detection	✓	—

Table 10: *Summary of implemented features and claimed grade contributions.*

7. Balance Sheet & Reflection

7.1. Work Distribution

The following is the estimated work distribution in the group for this implementation project.

	A. Ursachi	M. Á. Simonet S.	S. F. K. Abdul M.
Basic colour refinement	40%	30%	30%
Fast partition refinement	50%	25%	25%
Branching algorithm	30%	35%	35%
Generating sets / pruning	30%	40%	30%
Tree/forest preprocessing	25%	25%	50%
Preprocessing (twins)	35%	35%	30%
Documentation	33%	33%	34%

Table 11: *Work Distribution Group 2.*

7.2. Team Dynamics

+ Work distribution was fair and balanced across all three members.

+ Interdisciplinary perspectives: group members with stronger mathematical backgrounds helped formalise correctness arguments, while those with more programming experience optimised data structures and performance.

+ Code reviews and pair programming sessions helped transfer knowledge about each module's implementation details.

- Coordinating schedules for group meetings was occasionally challenging due to different timetables.

- Integrating independently developed modules (e.g., twin reduction with the main branching pipeline) required careful interface design and debugging.

7.3. Development process

All algorithms were implemented based on the reader and lecture materials.

References

[1] Hopcroft J. (1971) An $n \log n$ algorithm for minimizing states in a finite automaton. In *Theory of machines and computations* (Proc. Internat. Sympos., Technion, Haifa, 1971), New York: Academic Press, 189–196.

[2] J. Boon, W. Fokkema, R. Hoeksma, B. Kanger, D. van Kats. Implementation Project Graph Isomorphism. Faculty EEMCS, University of Twente, 2026.

A. Source Code: graph.py

This module provides the `Graph`, `Vertex`, and `Edge` classes. It was provided as part of the project framework.

```python
"""
This is a module for working with directed and undirected multigraphs.
"""
from typing import List, Union, Set


class GraphError(Exception):
    """
    An error that occurs while manipulating a `Graph`
    """

    def __init__(self, message: str):
        """
        Constructor
        :param message: The error message
        :type message: str
        """
        super(GraphError, self).__init__(message)


class Vertex(object):
    """
    `Vertex` objects have a property `graph` pointing to the graph they are part of,
    and an attribute `label` which can be anything: it is not used for any methods,
    except for `__str__`.
    """

    def __init__(self, graph: "Graph", label=None):
        """
        Creates a vertex, part of `graph`, with optional label `label`.
        (Labels of different vertices may be chosen the same; this does
        not influence correctness of the methods, but will make the string
        representation of the graph ambiguous.)
        :param graph: The graph that this `Vertex` is a part of
        :param label: Optional parameter to specify a label for the
        """
        if label is None:
            label = graph._next_label()

        self._graph = graph
        self.label = label
        self._incidence = {}

    def __repr__(self):
        """
        A programmer-friendly representation of the vertex.
```

```
        :return: The string to approximate the constructor arguments of the ‘Vertex’
        """
        return 'Vertex(label={}, #incident={})'.format(self.label, len(self._incidence))

    def __str__(self) -> str:
        """
        A user-friendly representation of the vertex, that is, its label.
        :return: The string representation of the label.
        """
        return str(self.label)

    def is_adjacent(self, other: "Vertex") -> bool:
        """
        Returns True iff ‘self‘ is adjacent to ‘other‘ vertex.
        :param other: The other vertex
        """
        return other in self._incidence

    def _add_incidence(self, edge: "Edge"):
        """
        For internal use only; adds an edge to the incidence map
        :param edge: The edge that is used to add the incidence
        """
        other = edge.other_end(self)

        if other not in self._incidence:
            self._incidence[other] = set()

        self._incidence[other].add(edge)

    @property
    def graph(self) -> "Graph":
        """
        The graph of this vertex
        :return: The graph of this vertex
        """
        return self._graph

    @property
    def incidence(self) -> List["Edge"]:
        """
        Returns the list of edges incident with the vertex.
        :return: The list of edges incident with the vertex
        """
        result = set()

        for edge_set in self._incidence.values():
            result |= edge_set

        return list(result)

    @property
    def neighbours(self) -> List["Vertex"]:
        """
        Returns the list of neighbors of the vertex.
        """
        return list(self._incidence.keys())

    @property
    def degree(self) -> int:
        """
        Returns the degree of the vertex
        """
        return sum(map(len, self._incidence.values()))


class Edge(object):
    """
    Edges have properties ‘tail‘ and ‘head‘ which point to the end vertices
    (‘Vertex‘ objects). The order of these matters when the graph is directed.
    """

```

```python
    def __init__(self, tail: Vertex, head: Vertex, weight=None):
        """
        Creates an edge between vertices `tail` and `head`
        :param tail: In case the graph is directed, this is the tail of the arrow.
        :param head: In case the graph is directed, this is the head of the arrow.
        :param weight: Optional weight of the vertex, which can be any type, but usually is a number.
        """
        if tail.graph != head.graph:
            raise GraphError("Can only add edges between vertices of the same graph")

        self._tail = tail
        self._head = head
        self._weight = weight

    def __repr__(self):
        """
        A programmer-friendly representation of the edge.
        :return: The string to approximate the constructor arguments of the `Edge`
        """
        return 'Edge(head={}, tail={}, weight={})'.format(self.head.label, self.tail.label, self.weight)

    def __str__(self) -> str:
        """
        A user friendly representation of this edge
        :return: A user friendly representation of this edge
        """
        return '({}, {})'.format(str(self.tail), str(self.head))

    @property
    def tail(self) -> "Vertex":
        """
        In case the graph is directed, this represents the tail of the arrow.
        :return: The tail of this edge
        """
        return self._tail

    @property
    def head(self) -> "Vertex":
        """
        In case the graph is directed, this represents the head of the arrow.
        :return: The head of this edge
        """
        return self._head

    @property
    def weight(self):
        """
        The weight of this edge, which can also just be used as a generic label.
        :return: The weight of this edge
        """
        return self._weight

    def other_end(self, vertex: Vertex) -> Vertex:
        """
        Given one end `vertex` of the edge, this returns
        the other end vertex.
        :param vertex: One end
        :return: The other end
        """
        if self.tail == vertex:
            return self.head
        elif self.head == vertex:
            return self.tail

        raise GraphError(
            'edge.other_end(vertex): vertex must be head or tail of edge')

    def incident(self, vertex: Vertex) -> bool:
        """
        Returns True iff the edge is incident with the
```

```python
        vertex.
        :param vertex: The vertex
        :return: Whether the vertex is incident with the edge.
        """
        return self.head == vertex or self.tail == vertex


class Graph(object):
    def __init__(self, directed: bool, n: int=0, simple: bool=False):
        """
        Creates a graph.
        :param directed: Whether the graph should behave as a directed graph.
        :param simple: Whether the graph should be a simple graph, that is, not have multi-edges or loops.
        :param n: Optional, the number of vertices the graph should create immediately
        """

        # _v and _vlist contain the same vertices in different data structures.

        self._v = set()
        self._vlist = list()
        self._e = list()
        self._simple = simple
        self._directed = directed
        self._next_label_value = 0

        for i in range(n):
            self.add_vertex(Vertex(self))

    def __repr__(self):
        """
        A programmer-friendly representation of the Graph.
        :return: The string to approximate the constructor arguments of the 'Graph'
        """
        return 'Graph(directed={}, simple={}, #edges={n_edges}, #vertices={n_vertices})'.format(
            self._directed, self._simple, n_edges=len(self._e), n_vertices=len(self._v))

    def __str__(self) -> str:
        """
        A user-friendly representation of this graph
        :return: A textual representation of the vertices and edges of this graph
        """
        return 'V=[' + ", ".join(map(str, self._vlist)) + ']\nE=[' + ", ".join(map(str, self._e)) + ']'

    def _next_label(self) -> int:
        """
        Generates unique labels for vertices within the graph
        :return: A unique label
        """
        result = self._next_label_value
        self._next_label_value += 1
        return result

    @property
    def simple(self) -> bool:
        """
        Whether the graph is a simple graph, that is, it does not have multi-edges or loops.
        :return: Whether the graph is simple
        """
        return self._simple

    @property
    def directed(self) -> bool:
        """
        Whether the graph behaves as a directed graph
        :return: Whether the graph is directed
        """
        return self._directed

    @property
```

```python
    def vertices(self) -> List["Vertex"]:
        """
        :return: The ‘set‘ of vertices of the graph
        """
        return list(self._vlist)

    @property
    def edges(self) -> List["Edge"]:
        """
        :return: The ‘set‘ of edges of the graph
        """
        return list(self._e)

    def __iter__(self):
        """
        :return: Returns an iterator for the vertices of the graph
        """
        return iter(self._vlist)

    def __len__(self) -> int:
        """
        :return: The number of vertices of the graph
        """
        return len(self._vlist)

    def add_vertex(self, vertex: "Vertex"):
        """
        Add a vertex to the graph.
        :param vertex: The vertex to be added.
        """
        if vertex.graph != self:
            raise GraphError("A vertex must belong to the graph it is added to")

        self._v.add(vertex)
        self._vlist.append(vertex)

    def add_edge(self, edge: "Edge"):
        """
        Add an edge to the graph. And if necessary also the vertices.
        Includes some checks in case the graph should stay simple.
        :param edge: The edge to be added
        """

        if self._simple:
            if edge.tail == edge.head:
                raise GraphError('No loops allowed in simple graphs')

            if self.is_adjacent(edge.tail, edge.head):
                raise GraphError('No multiedges allowed in simple graphs')

        if edge.tail not in self._v:
            self.add_vertex(edge.tail)
        if edge.head not in self._v:
            self.add_vertex(edge.head)

        self._e.append(edge)

        edge.head._add_incidence(edge)
        edge.tail._add_incidence(edge)

    def __iadd__(self, other: Union[Edge, Vertex]) -> "Graph":
        """
        Add either an ‘Edge‘ or ‘Vertex‘ with the += syntax.
        :param other: The object to be added
        :return: The modified graph
        """
        if isinstance(other, Vertex):
            self.add_vertex(other)

        if isinstance(other, Edge):
            self.add_edge(other)

```

```python
        return self

    def find_edge(self, u: "Vertex", v: "Vertex") -> Set["Edge"]:
        """
        Tries to find edges between two vertices.
        :param u: One vertex
        :param v: The other vertex
        :return: The set of edges incident with both ‘u‘ and ‘v‘
        """
        result = u._incidence.get(v, set())

        if not self._directed:
            result |= v._incidence.get(u, set())

        return set(result)

    def is_adjacent(self, u: "Vertex", v: "Vertex") -> bool:
        """
        Returns True iff vertices ‘u‘ and ‘v‘ are adjacent. If the graph is directed, the direction of the edges is
        respected.
        :param u: One vertex
        :param v: The other vertex
        :return: Whether the vertices are adjacent
        """
        return v in u.neighbours and (not self.directed or any(e.head == v for e in u.incidence))


class UnsafeGraph(Graph):
    @property
    def vertices(self) -> List["Vertex"]:
        return self._v

    @property
    def edges(self) -> List["Edge"]:
        return self._e

    def add_vertex(self, vertex: "Vertex"):
        self._v.add(vertex)
        self._vlist.append(vertex)

    def add_edge(self, edge: "Edge"):
        self._e.append(edge)

        edge.head._add_incidence(edge)
        edge.tail._add_incidence(edge)

    def find_edge(self, u: "Vertex", v: "Vertex") -> Set["Edge"]:
        left = u._incidence.get(v, None)
        right = None

        if not self._directed:
            right = v._incidence.get(u, None)

        if left is None and right is None:
            return set()

        if left is None:
            return right

        if right is None:
            return left

        return left | right

    def is_adjacent(self, u: "Vertex", v: "Vertex") -> bool:
        return v in u._incidence or (not self._directed and u in v._incidence)
```

Listing 1: *graph.py — Graph data structure*

B. Source Code: graph_io.py

This module provides functions for reading and writing graphs in the .gr/.grl file format, as well as .dot and .graphML visualisation formats. It was provided as part of the project framework.

```python
"""
Includes functions for reading and writing graphs, in a very simple readable format.
"""
import sys
import colorsys
import xml.etree.ElementTree as ET
from typing import IO, Tuple, List, Union, Any
from collections import defaultdict
from graph import Graph, Edge


# GraphML settings
COLORBLIND_MODE = False  # Start with variations in value and saturation rather than hue
TYPE_TO_STRING = {str: 'string', int: 'int', bool: 'bool', float: 'float'}
DEFAULT_ATTRIBUTES_GRAPHML = {'node': {'color': 'light gray', 'label': str, 'size': 30}, 'edge': {'color': 'black'}}

# Dot settings
NUM_COLORS = 12  # Make combinations of fillcolor and edgecolor in order to get more distinct nodes
USE_GRAPHML_COLORS = False  # Overwrite the .dot file color scheme by the graphML color scheme.
DEFAULT_ATTRIBUTES_DOT = {'node': {'penwidth': 1 if USE_GRAPHML_COLORS else 4, 'label': str, 'color': str, 'width': .5,
                                   'height': .5, 'margin': .05, 'style': 'filled', 'colorscheme': 'paired12'},
                          'edge': {'penwidth': 2}}


def num_to_color(n: int) -> str:
    """
    Converts natural numbers to colors such that numbers that are close together map to distinguishable colors.
    :param n: Natural number
    :return: Color in hex format as a string
    """
    n = int(n)
    sv_combs = [(0.8, 1), (0.2, 1), (0.8, 0.7)]

    if COLORBLIND_MODE:
        s, v = sv_combs[n % len(sv_combs)]
        hue_6 = (n//len(sv_combs)) % 6
    else:
        hue_6 = n % 6
        s, v = sv_combs[(n//6) % len(sv_combs)]

    n //= 6 * len(sv_combs)

    hue_rem = bin(n)[:1:-1]
    hue_frac = int(hue_rem, 2) / (1 << len(hue_rem))
    h = 1 - (hue_6 + hue_frac) / 6

    (r, g, b) = colorsys.hsv_to_rgb(h, s, v)
    (r, g, b) = (round(255*r), round(255*g), round(255*b))

    return f'#{r:02x}{g:02x}{b:02x}'


def read_line(f: IO[str]) -> str:
    """
    Read a single non-comment line from a file
    :param f: The file
    :return: The line
    """
    line = f.readline()

```

```python
    while len(line) > 0 and line[0] == '#':
        line = f.readline()

    return line


def read_graph(graphclass, f: IO[str]) -> Tuple[Graph, bool]:
    """
    Read a graph from a file
    :param graphclass: The class of the graph
    :param f: The file
    :return: The graph
    """
    while True:
        try:
            line = read_line(f)
            n = int(line)
            graph = graphclass(directed=False, n=n)
            break
        except ValueError:
            pass

    line = read_line(f)
    edges = []

    try:
        while True:
            comma = line.find(',')
            if ':' in line:
                colon = line.find(':')
                edges.append((int(line[:comma]), int(line[comma + 1:colon]), int(line[colon + 1:])))
            else:
                edges.append((int(line[:comma]), int(line[comma + 1:]), None))
            line = read_line(f)
    except:
        pass

    indexed_nodes = list(graph.vertices)

    for edge in edges:
        graph += Edge(indexed_nodes[edge[0]], indexed_nodes[edge[1]], edge[2])

    if line and line[0] == '-':
        return graph, True
    else:
        return graph, False


def read_graph_list(graph_class, f: IO[str]) -> List[Graph]:
    """
    Read a list of graphs from a file
    :param graph_class: The graph class
    :param f: The file
    :return: A list of graphs
    """
    graphs = []
    cont = True

    while cont:
        graph, cont = read_graph(graph_class, f)
        graphs.append(graph)

    return graphs


def load_graph(f: IO[str], graph_class=Graph, read_list: bool = False) -> Union[List[Graph], Graph]:
    """
    Load a graph from a file
    :param f: The file
    :param graph_class: Class of the graph. You may subclass the default graph class and add
```

```
        your own here.
    :param read_list: Specifies whether to read a list of graphs from the file, or just a single graph.
    :return: The graph, or a list of graphs.
    """
    if read_list:
        graph_list = read_graph_list(graph_class, f)
        return graph_list
    else:
        graph, _ = read_graph(graph_class, f)
        return graph


def input_graph(graph_class=Graph, read_list: bool = False) -> Union[Tuple[List[Graph], List[str]], Graph]:
    """
    Load a graph from sys.stdin
    :param graph_class: The class of the graph. You may subclass the default graph class and add your own here.
    :param read_list: Specifies whether to read a list of graphs from the file, or just a single graph.
    :return: The graph, or a list of graphs.
    """
    return load_graph(f=sys.stdin, graph_class=graph_class, read_list=read_list)


def write_line(f: IO[str], line: str):
    """
    Write a line to a file
    :param f: The file
    :param line: The line
    """
    f.write(line + '\n')


def write_graph_list(graph_list: List[Graph], f: IO[str]):
    """
    Write a graph list to a file.
    :param graph_list: List of graphs
    :param f: The file
    """
    for i, g in enumerate(graph_list):
        n = len(g)
        write_line(f, '# Number of vertices:')
        write_line(f, str(n))

        # Give the vertices (temporary) labels from 0 to n-1:
        label = {}
        for vertex_index, vertex in enumerate(g):
            label[vertex] = vertex_index

        write_line(f, '# Edge list:')

        for e in g.edges:
            if e.weight:
                write_line(f, str(label[e.tail]) + ',' + str(label[e.head]) + ':' + str(e.weight))
            else:
                write_line(f, str(label[e.tail]) + ',' + str(label[e.head]))

        if i + 1 < len(graph_list):
            write_line(f, '--- Next graph:')


def save_graph(graph_list: Union[Graph, List[Graph]], f: IO[str]):
    """
    Write a graph, or a list of graphs to a file.
    :param graph_list: The graph, or a list of graphs.
    :param f: The file
    """
    if type(graph_list) is list:
        write_graph_list(graph_list, f)
```

```
    else:
        write_graph_list([graph_list], f)


def print_graph(graph_list: Union[Graph, List[Graph]]):
    """
    Print a graph, or a list of graphs to sys.stdout
    :param graph_list: The graph, or list of graphs.
    """
    if type(graph_list) is list:
        write_graph_list(graph_list, sys.stdout)
    else:
        write_graph_list([graph_list], sys.stdout)


def get_attribute_value(attribute: str, obj, dict_or_default_or_type):
    """
    Helper function for the write_graphml and write_dot functions. Tries to determine whether
        obj has a given attribute.
    :param attribute:
    :param obj:
    :param dict_or_default_or_type:
    :return:
    """
    if isinstance(dict_or_default_or_type, dict):
        if obj in dict_or_default_or_type:
            return dict_or_default_or_type[obj]
        else:
            return None
    elif hasattr(obj, attribute):
        return getattr(obj, attribute)
    elif type(dict_or_default_or_type) is type:
        return None
    return dict_or_default_or_type


def write_graphml(graph: Graph, f: IO[str], node_attributes: dict[str, Any] = None,
                  edge_attributes: dict[str, Any] = None):
    """
    Writes a graph to a .graphML file. Look at the documentation of write_visualization for
        more information.

    :param graph: The graph
    :param f: The file
    :param node_attributes: Optional node attributes to write to the file
    :param edge_attributes: Optional edge attributes to write to the file
    """

    if node_attributes is None:
        node_attributes = {}
    else:
        node_attributes = node_attributes.copy()
    if edge_attributes is None:
        edge_attributes = {}
    else:
        edge_attributes = edge_attributes.copy()

    for attribute, default in DEFAULT_ATTRIBUTES_GRAPHML['node'].items():
        if attribute not in node_attributes:
            node_attributes[attribute] = default

    for attribute, default in DEFAULT_ATTRIBUTES_GRAPHML['edge'].items():
        if attribute not in edge_attributes:
            edge_attributes[attribute] = default

    root = ET.Element('graphML')

    object_attribute_to_key = defaultdict(lambda: 'd'+str(len(object_attribute_to_key)))

    for object_type, object_attributes in [('node', node_attributes), ('edge',
        edge_attributes)]:
        for attribute in object_attributes:
```

```
            if attribute == 'color' and object_type == 'node':
                attribute_type = str
            elif isinstance(object_attributes[attribute], dict):
                attribute_type = type(next(iter(object_attributes[attribute].values())))
            elif type(object_attributes[attribute]) is type:
                attribute_type = object_attributes[attribute]
            else:
                attribute_type = type(object_attributes[attribute])

            if not any(isinstance(attribute, t) for t in TYPE_TO_STRING):
                raise ValueError(f"Attributes may only have type int, bool, str or float, not {type(attribute_type)}")

            ET.SubElement(root, 'key', {'id': object_attribute_to_key[(object_type, attribute)],
                                        'for': object_type,
                                        'attr.name': attribute,
                                        'attr.type': TYPE_TO_STRING[attribute_type]})

    edgedefault = 'directed' if graph.directed else 'undirected'
    graph_obj = ET.SubElement(root, 'graph', {'id': str(id(graph)), 'edgedefault': edgedefault})

    for v in graph:
        node = ET.SubElement(graph_obj, 'node', {'id': str(id(v))})

        for attribute in node_attributes:
            value = get_attribute_value(attribute, v, node_attributes[attribute])
            if value is None:
                continue
            if attribute == 'color' and isinstance(value, int):
                value = num_to_color(value)

            nc = ET.SubElement(node, 'data', {'key': object_attribute_to_key[('node', attribute)]})
            nc.text = str(value)

    for e in graph.edges:
        edge = ET.SubElement(graph_obj, 'edge', {'source': str(id(e.tail)), 'target': str(id(e.head))})

        for attribute in edge_attributes:
            value = get_attribute_value(attribute, e, edge_attributes[attribute])
            if value is None:
                continue

            nc = ET.SubElement(edge, 'data', {'key': object_attribute_to_key[('edge', attribute)]})
            nc.text = str(value)

    ET.indent(root)
    s = ET.tostring(root, encoding='unicode')
    f.write(s)


def write_dot(graph: Graph, f: IO[str], node_attributes: dict[str, Any] = None,
              edge_attributes: dict[str, Any] = None):
    """
    Writes a graph to a .dot file. Look at the documentation of write_visualization for more information.

    :param graph: The graph
    :param f: The file
    :param node_attributes: Optional node attributes to write to the file
    :param edge_attributes: Optional edge attributes to write to the file
    """

    if node_attributes is None:
        node_attributes = {}
    else:
        node_attributes = node_attributes.copy()
    if edge_attributes is None:
```

```
        edge_attributes = {}
    else:
        edge_attributes = edge_attributes.copy()

    for attribute, default in DEFAULT_ATTRIBUTES_DOT['node'].items():
        if attribute not in node_attributes:
            node_attributes[attribute] = default

    for attribute, default in DEFAULT_ATTRIBUTES_DOT['edge'].items():
        if attribute not in edge_attributes:
            edge_attributes[attribute] = default

    if graph.directed:
        f.write('digraph G {\n')
        edge_symbol = ' -> '
    else:
        f.write('graph G {\n')
        edge_symbol = ' -- '

    f.write('node [')

    for attribute, default in DEFAULT_ATTRIBUTES_DOT['node'].items():
        if not isinstance(default, dict) and not type(default) is type:
            if isinstance(default, str):
                f.write(f'{attribute}="{default}",')
            else:
                f.write(f'{attribute}={str(default)},')

    f.write(']\nedge [')
    for attribute, default in DEFAULT_ATTRIBUTES_DOT['edge'].items():
        if not isinstance(default, dict) and not type(default) is type:
            if isinstance(default, str):
                f.write(f'{attribute}="{default}",')
            else:
                f.write(f'{attribute}={str(default)},')

    f.write(']\n')

    v_to_name = defaultdict(lambda: len(v_to_name))

    for v in graph:
        f.write(f'{v_to_name[v]} [')
        for attribute in node_attributes:
            value = get_attribute_value(attribute, v, node_attributes[attribute])
            if value is None or value is node_attributes[attribute]:
                continue
            if attribute == 'color' and isinstance(value, int):
                if USE_GRAPHML_COLORS:
                    f.write(f'fillcolor="{num_to_color(value)}",')
                    continue
                if value > NUM_COLORS:
                    fillcolor = (value // NUM_COLORS) % NUM_COLORS + 1
                    f.write(f'fillcolor={str(fillcolor)},')
                value = value % NUM_COLORS + 1
            if isinstance(value, str):
                f.write(f'{attribute}="{value}",')
            else:
                f.write(f'{attribute}={value},')
        f.write(']\n')

    f.write('\n')

    for e in graph.edges:
        f.write(f'{v_to_name[e.tail]} {edge_symbol} {v_to_name[e.head]} [')
        for attribute in edge_attributes:
            value = get_attribute_value(attribute, e, edge_attributes[attribute])
            if value is None or value is edge_attributes[attribute]:
                continue
            if isinstance(value, str):
                f.write(f'{attribute}="{value}",')
            else:
                f.write(f'{attribute}={value},')
```

```python

        f.write(']\n')

    f.write('}')


def write_visualization(graph: Graph, f: IO[str], node_attributes: dict[str, Any] = None,
                        edge_attributes: dict[str, Any] = None):
    """
    Writes a graph to the given file in .graphML or .dot format, matching the extension of the file.
    Optional attributes for the nodes and edges can be given in node_attributes and edge_attributes.

    Attributes can be given as a dict that maps vertices or edges to their corresponding values. Alternatively, it is
    possible to give a default value or type and look for the attributes by name in the vertex and edge objects.

    Example for node_attributes:

    node_attributes = {'color': {<v_1> : 1, <v_2> : 2, <v_4>: 1, ...}, # Only vertices in the dictionary get a value
                       'size': 30,  # Look for <v>.size, otherwise write a default value of 30
                       'weight': int  # Look for <v>.weight, otherwise do not write a value
                       }

    Depending on the file format, some attributes will be written by default from the DEFAULT_ATTRIBUTES_ constant.
    A list of attributes that might be used by .dot viewers can be found at https://graphviz.org/doc/info/attrs.html
    Gephi only uses the following graphML attributes by default for the visualization: 'label', 'color',
    'weight' (edges only) and 'x', 'y', 'z', 'size' for nodes only. Gephi cannot read the .dot files.

    :param graph: The graph
    :param f: The file
    :param node_attributes: Optional node attributes to write to the file
    :param edge_attributes: Optional edge attributes to write to the file
    """
    if f.name.endswith('.dot'):
        write_dot(graph, f, node_attributes, edge_attributes)
    elif f.name.endswith('.graphML'):
        write_graphml(graph, f, node_attributes, edge_attributes)
    else:
        raise ValueError(f"Writing visualizations is only supported for file extensions '.dot' and '.graphML', "
                         f"so {f.name} is invalid filename.")


if __name__ == "__main__":
    with open('examplegraph.gr') as file:
        G = load_graph(file, Graph)
        print(G)
```

Listing 2: *graph_io.py — Graph I/O utilities*

C. Source Code: colorref.py

Basic colour refinement ($O(n^2m)$ Weisfeiler-Lehman 1D algorithm). Implements Algorithm 1 from the reader (Chapter 2).

```python
"""
Basic Colour Refinement -- O(n²m) Weisfeiler-Lehman 1D algorithm.

Iteratively refines vertex colours based on the sorted multiset of
```

```python
neighbour colours until a stable colouring is reached (no colour class
splits further).  All graphs are processed simultaneously in a single
global colour space (disjoint union) so that isomorphic graphs receive
identical colour signatures.

Reference: Chapter 2 of the reader (Algorithm 1 -- Colour Refinement).
"""

from collections import defaultdict
from typing import List, Dict, Tuple
from graph import Graph, Vertex
from graph_io import load_graph


# ---------------------------------------------------------------------------
# Initialisation
# ---------------------------------------------------------------------------

def set_default_colouring(graph: Graph, mode: str = "uniform") -> None:
    """Assign an initial colour to every vertex in *graph*.

    Args:
        graph:  The graph whose vertices are coloured in-place.
        mode:   ``"uniform"`` sets colour 1 for all vertices.
                ``"degree"`` sets each vertex's colour to its degree.
    """
    for v in graph:
        v.colour = v.degree if mode == "degree" else 1


# ---------------------------------------------------------------------------
# Partition helpers
# ---------------------------------------------------------------------------

def partition_vertices(vertices: List[Vertex]) -> Dict[Tuple, List[Vertex]]:
    """Group vertices by their colour signature.

    A vertex's signature is the tuple
    ``(own_colour, sorted_neighbour_colour_1, ..)``.
    Vertices with the same signature are structurally indistinguishable
    by 1-WL and belong to the same partition cell.

    Args:
        vertices:  All vertices across every graph (disjoint union).

    Returns:
        A dict mapping each signature to its list of vertices.
    """
    partition: Dict[Tuple, List[Vertex]] = defaultdict(list)
    for v in vertices:
        signature = (v.colour, *sorted(n.colour for n in v.neighbours))
        partition[signature].append(v)
    return partition


def get_sorted_partition(vertices: List[Vertex]) -> List[List[Vertex]]:
    """Return a deterministically sorted list of partition cells.

    Sorting by signature ensures that isomorphic graphs receive the
    exact same new colour ids in the next refinement step.

    Args:
        vertices:  All vertices across every graph.

    Returns:
        A list of vertex groups, sorted by their signature.
    """
    partition = partition_vertices(vertices)
    return [cell for _, cell in sorted(partition.items())]


# ---------------------------------------------------------------------------
```

```python
# Refinement step
# ---------------------------------------------------------------------------

def assign_colours_from_partition(partition: List[List[Vertex]]) -> None:
    """Assign new integer colours based on partition position.

    Cell *i* in the sorted partition receives colour *i*.
    Modifies vertex colours in-place.

    Args:
        partition:  Sorted list of vertex groups (from ``get_sorted_partition``).
    """
    for colour, cell in enumerate(partition):
        for v in cell:
            v.colour = colour


# ---------------------------------------------------------------------------
# Public API
# ---------------------------------------------------------------------------

def basic_colorref(filename: str) -> List[Tuple[List[int], List[int], int, bool]]:
    """Run basic 1-WL colour refinement on a ``.grl`` file.

    Reads all graphs from the file, pools their vertices into a single
    disjoint union, and iteratively refines until the global colouring
    is stable.  Graphs are then grouped into equivalence classes by
    their sorted colour multiset.

    Args:
        filename:  Path to a ``.grl`` file containing one or more graphs.

    Returns:
        A sorted list of tuples, one per equivalence class:
            - ``graph_indices`` -- sorted list of 0-based graph indices.
            - ``colour_class_sizes`` -- sorted list of colour-class sizes.
            - ``iterations`` -- number of refinement rounds to stability.
            - ``is_discrete`` -- whether every colour class has size 1.
    """
    with open(filename, "r") as file:
        graphs = load_graph(file, graph_class=Graph, read_list=True)

    # 1. Pool all vertices and assign uniform initial colouring
    all_vertices: List[Vertex] = []
    for G in graphs:
        set_default_colouring(G)
        all_vertices.extend(G)

    # Per-graph colour counts (used to track individual convergence)
    num_colours_per_graph = [len({v.colour for v in G}) for G in graphs]
    iterations_to_stability = [0] * len(graphs)

    global_num_colours = len({v.colour for v in all_vertices})
    iteration = 0

    # 2. Iterative refinement loop
    while True:
        current_partition = get_sorted_partition(all_vertices)

        # Stop when the number of global colour classes no longer increases
        if global_num_colours == len(current_partition):
            break

        assign_colours_from_partition(current_partition)
        global_num_colours = len(current_partition)
        iteration += 1

        # Track the iteration at which each graph individually stabilises
        for i, G in enumerate(graphs):
            current_g_colours = len({v.colour for v in G})
            if current_g_colours > num_colours_per_graph[i]:
                num_colours_per_graph[i] = current_g_colours
```

```
                iterations_to_stability[i] = iteration

    # 3. Group graphs by their global colour signature
    graph_classes: Dict[Tuple, List[int]] = defaultdict(list)
    for i, G in enumerate(graphs):
        signature = tuple(sorted(v.colour for v in G))
        graph_classes[signature].append(i)

    # 4. Format output
    result: List[Tuple[List[int], List[int], int, bool]] = []
    for signature, indices in graph_classes.items():
        sorted_indices = sorted(indices)

        # Reconstruct colour-class sizes from the signature
        counts: Dict[int, int] = defaultdict(int)
        for colour in signature:
            counts[colour] += 1
        sizes = sorted(counts.values())

        iters = iterations_to_stability[sorted_indices[0]]
        is_discrete = len(sizes) == sum(sizes)

        result.append((sorted_indices, sizes, iters, is_discrete))

    return sorted(result, key=lambda x: x[0][0])
```

Listing 3: *colorref.py — Basic Colour Refinement*

D. Source Code: fast_colorref.py

Fast colour refinement ($O(m \log n)$ Hopcroft-style partition refinement). Implements the algorithm from Chapter 4 of the reader.

```
"""
Fast Colour Refinement -- O(m log n) Hopcroft-style partition refinement.

Based on the DFA minimisation algorithm (Hopcroft, 1971) adapted for
the Graph Isomorphism problem as described in Chapter 4 of the reader.

Refine(C) for GI:  For each colour class D, partition D by the number
of neighbours each vertex has inside C.  If D splits, apply the
"add-smaller-to-queue" rule (Lemma 4.9).
"""

from collections import deque
from typing import Dict, List, Tuple, Optional
from graph import Graph, Vertex


# ---------------------------------------------------------------------------
# Data structures
# ---------------------------------------------------------------------------

class Partition:
    """Partition of vertices into colour classes.

    ``classes[colour]`` is a ``set`` of vertices.
    ``vertex_colour[v]`` gives the colour of vertex v.
    """

    __slots__ = ("classes", "vertex_colour", "_next_colour")

    def __init__(self):
        self.classes: Dict[int, set] = {}
        self.vertex_colour: Dict = {}
        self._next_colour: int = 0

    def add_class(self, members: set) -> int:
```

```python
        """Create a new colour class from *members* and return its colour id.

        Each vertex in *members* is assigned the new colour in
        ``vertex_colour``.
        """
        c = self._next_colour
        self._next_colour += 1
        self.classes[c] = members
        vc = self.vertex_colour
        for v in members:
            vc[v] = c
        return c

    @property
    def num_classes(self) -> int:
        return len(self.classes)


# ---------------------------------------------------------------------------
# Adjacency helper
# ---------------------------------------------------------------------------

def _build_neighbour_index(vertices) -> Dict:
    """Pre-compute adjacency lists for fast iteration.

    Returns a dict mapping each vertex to its list of neighbours.
    Caching this avoids repeated property lookups during refinement.

    Args:
        vertices:  Iterable of Vertex objects (may span multiple graphs).

    Returns:
        Dict mapping vertex → list of neighbour vertices.
    """
    return {v: v.neighbours for v in vertices}


# ---------------------------------------------------------------------------
# Core algorithm
# ---------------------------------------------------------------------------

def fast_colour_refine(vertices: list,
                       adj: Dict,
                       initial_colouring: Dict) -> Partition:
    """Run fast colour refinement to a stable partition.

    Args:
        vertices:  All vertices (disjoint union).
        adj:       vertex → neighbour list.
        initial_colouring:  vertex → initial colour.

    Returns:
        The stable Partition.
    """
    partition = Partition()

    # Build initial partition
    groups: Dict[int, set] = {}
    for v in vertices:
        c = initial_colouring[v]
        if c in groups:
            groups[c].add(v)
        else:
            groups[c] = {v}

    queue: deque = deque()
    in_queue: set = set()
    for _, members in sorted(groups.items()):
        colour = partition.add_class(members)
        queue.append(colour)
        in_queue.add(colour)

```

```
    # Local aliases
    p_classes = partition.classes
    p_vc = partition.vertex_colour
    p_nc = [partition._next_colour]  # mutable ref for inline splits

    while queue:
        rc = queue.popleft()
        in_queue.discard(rc)

        refine_members = p_classes.get(rc)
        if refine_members is None:
            continue

        # === Refine(C) -- tight inlined loop ===

        # Count how many neighbours each vertex has in C.
        neighbour_count: Dict = {}
        for u in refine_members:
            for w in adj[u]:
                if w in neighbour_count:
                    neighbour_count[w] += 1
                else:
                    neighbour_count[w] = 1

        # Group affected vertices by their current colour class.
        affected_by_colour: Dict[int, list] = {}
        for w in neighbour_count:
            wc = p_vc[w]
            if wc in affected_by_colour:
                affected_by_colour[wc].append(w)
            else:
                affected_by_colour[wc] = [w]

        # For each affected colour class, split if vertices disagree on count.
        for d_colour, affected_list in affected_by_colour.items():
            d_members = p_classes.get(d_colour)
            if d_members is None:
                continue

            d_size = len(d_members)
            affected_count = len(affected_list)

            # Group affected vertices by their count
            count_groups: Dict[int, list] = {}
            for v in affected_list:
                cnt = neighbour_count[v]
                if cnt in count_groups:
                    count_groups[cnt].append(v)
                else:
                    count_groups[cnt] = [v]

            unaffected_count = d_size - affected_count

            # How many distinct groups?
            num_groups = len(count_groups)
            if unaffected_count > 0:
                num_groups += 1
            if num_groups <= 1:
                continue

            # Find largest fragment to keep in original colour
            largest_key = 0  # 0 means the unaffected group
            largest_size = unaffected_count

            for cnt, grp in count_groups.items():
                gs = len(grp)
                if gs > largest_size:
                    largest_size = gs
                    largest_key = cnt

            # Perform splits
            d_in_queue = d_colour in in_queue
```

```
            nc_val = p_nc[0]

            if largest_key == 0:
                # Keep unaffected in original; split out all count groups
                for cnt, grp in count_groups.items():
                    grp_set = set(grp)
                    d_members -= grp_set
                    p_classes[nc_val] = grp_set
                    for v in grp:
                        p_vc[v] = nc_val
                    if d_in_queue or nc_val not in in_queue:
                        queue.append(nc_val)
                        in_queue.add(nc_val)
                    nc_val += 1
            else:
                # Keep the largest count group; split out rest
                if unaffected_count > 0:
                    unaff = d_members - set(affected_list)
                    d_members -= unaff
                    p_classes[nc_val] = unaff
                    for v in unaff:
                        p_vc[v] = nc_val
                    if d_in_queue or nc_val not in in_queue:
                        queue.append(nc_val)
                        in_queue.add(nc_val)
                    nc_val += 1

                for cnt, grp in count_groups.items():
                    if cnt == largest_key:
                        continue
                    grp_set = set(grp)
                    d_members -= grp_set
                    p_classes[nc_val] = grp_set
                    for v in grp:
                        p_vc[v] = nc_val
                    if d_in_queue or nc_val not in in_queue:
                        queue.append(nc_val)
                        in_queue.add(nc_val)
                    nc_val += 1

            p_nc[0] = nc_val

    partition._next_colour = p_nc[0]
    return partition


# ---------------------------------------------------------------------------
# Public API
# ---------------------------------------------------------------------------

def colour_refine_graphs(graphs: List[Graph],
                         initial_colouring: Optional[Dict] = None
                         ) -> Partition:
    """Run fast colour refinement on a list of graphs (disjoint union).

    Pools all vertices from every graph into a single vertex set and
    refines them together.  Isomorphic graphs will end up with
    identical colour distributions.

    Args:
        graphs:             List of Graph objects to refine together.
        initial_colouring:  Optional vertex → colour dict.  If None,
                            all vertices start with colour 0.

    Returns:
        The stable Partition after refinement.
    """
    all_vertices = [v for G in graphs for v in G]
    if initial_colouring is None:
        initial_colouring = {v: 0 for v in all_vertices}
    adj = _build_neighbour_index(all_vertices)
    return fast_colour_refine(all_vertices, adj, initial_colouring)
```

```python


def get_colour_signature(graph: Graph, partition: Partition) -> Tuple:
    """Compute a canonical colour signature for *graph*.

    The signature is a sorted tuple of ``(colour, count)`` pairs
    describing how many vertices of each colour the graph has.
    Two graphs with different signatures are guaranteed non-isomorphic.

    Args:
        graph:      The graph to characterise.
        partition:  A stable Partition (from colour refinement).

    Returns:
        A hashable tuple suitable for dict keys / equality checks.
    """
    counts: Dict[int, int] = {}
    vc = partition.vertex_colour
    for v in graph:
        c = vc[v]
        counts[c] = counts.get(c, 0) + 1
    return tuple(sorted(counts.items()))


def is_balanced(graphs: List[Graph], partition: Partition) -> bool:
    """Check whether all graphs share the same colour distribution.

    A balanced colouring is a necessary (but not sufficient) condition
    for isomorphism.  If unbalanced, the graphs are definitely not
    isomorphic.

    Args:
        graphs:     List of Graph objects to compare.
        partition:  A stable Partition.

    Returns:
        True if every graph has the same colour signature.
    """
    if len(graphs) < 2:
        return True
    ref = get_colour_signature(graphs[0], partition)
    for G in graphs[1:]:
        if get_colour_signature(G, partition) != ref:
            return False
    return True


def is_discrete_for_graph(graph: Graph, partition: Partition) -> bool:
    """Check if every vertex in *graph* has a unique colour.

    A discrete colouring means each colour class has exactly one
    vertex from this graph, so the partition fully distinguishes
    every vertex.

    Args:
        graph:      The graph to check.
        partition:  A stable Partition.

    Returns:
        True if every vertex in *graph* has a distinct colour.
    """
    n = len(graph)
    seen = set()
    vc = partition.vertex_colour
    for v in graph:
        seen.add(vc[v])
    return len(seen) == n


def defines_bijection(graph_a: Graph, graph_b: Graph,
                      partition: Partition) -> bool:
    """Check if the partition defines a unique bijection between two graphs.
```

```python

    This is the case when the colouring is both discrete and balanced:
    each vertex in graph_a can be uniquely matched to the vertex in
    graph_b with the same colour.

    Args:
        graph_a:    First graph.
        graph_b:    Second graph.
        partition:  A stable Partition.

    Returns:
        True if the partition is discrete for both graphs.
    """
    return (is_discrete_for_graph(graph_a, partition) and
            is_discrete_for_graph(graph_b, partition))


def basic_colorref_fast(filename: str) -> List[Tuple[List[int], List[int], int, bool]]:
    """Drop-in replacement for ``basic_colorref`` using fast colour refinement.

    Reads graphs from a ``.grl`` file and partitions them into
    equivalence classes based on colour signatures, using the O(m log n)
    algorithm instead of the O(n^2 m) basic version.

    Args:
        filename:  Path to a ``.grl`` file.

    Returns:
        Same format as ``basic_colorref``: a sorted list of tuples
        ``(graph_indices, colour_class_sizes, iterations, is_discrete)``.
    """
    from graph_io import load_graph
    with open(filename, 'r') as file:
        graphs = load_graph(file, graph_class=Graph, read_list=True)
    all_vertices = [v for G in graphs for v in G]
    initial_colouring = {v: 0 for v in all_vertices}
    adj = _build_neighbour_index(all_vertices)
    partition = fast_colour_refine(all_vertices, adj, initial_colouring)

    eq_classes: Dict[Tuple, List[int]] = {}
    for i, G in enumerate(graphs):
        sig = get_colour_signature(G, partition)
        if sig in eq_classes:
            eq_classes[sig].append(i)
        else:
            eq_classes[sig] = [i]

    result = []
    for sig, indices in eq_classes.items():
        sizes = sorted(count for _, count in sig)
        result.append((sorted(indices), sizes, 0, all(s == 1 for s in sizes)))
    return sorted(result, key=lambda x: x[0][0])
```

Listing 4: *`fast_colorref.py` — Fast Colour Refinement*

E. Source Code: `branching.py`

Branching (individualization-refinement) for GI and #Aut. Implements Algorithms 2 and 9 from the reader (Chapters 3 and 5), with Lemma 5.11 pruning and generating-set computation.

```python
"""
Branching -- Individualization-Refinement for GI and #Aut.

When fast colour refinement yields a stable colouring that is balanced
but not discrete, branching resolves the ambiguity by picking a vertex
x in graph G from a non-trivial colour class, trying every possible
mapping x → y in the same colour class of graph H, and recursing.

```

```python
Two branching strategies are provided:

  * ``count_isomorphisms``  (Algorithm 2, Chapter 3)
    Enumerates all isomorphisms -- used for GI and as a fallback.

  * ``count_automorphisms`` (Algorithm 9, Chapter 5)
    Builds a *generating set* of Aut(G) and computes its order via
    the orbit-stabiliser theorem.  Prunes redundant branches with
    Lemma 5.11, making it feasible even for graphs with 10^33+
    automorphisms (e.g. bigtrees3).

Key optimisations beyond the textbook algorithms:

  * **Degree-based initial colouring**: vertex degree is used as the
    initial colour (instead of uniform), giving colour refinement a
    head start.
  * **Highest-degree vertex selection**: within each branching class,
    the most constrained vertex (highest degree) is individualized
    first to maximise refinement propagation.
  * **Orbit pruning for GI**: before pairwise isomorphism tests,
    Aut(H) is computed for small graphs (<= 100 vertices).  At each
    branching level, the stabiliser of already-individualized vertices
    is used to compute orbits within the current colour class; only one
    representative per orbit is tried, drastically cutting the
    branching factor for symmetric graphs.
"""

from collections import defaultdict
from typing import List, Dict, Tuple, Optional
from graph import Graph, Vertex, Edge
from graph_io import load_graph
from fast_colorref import (
    colour_refine_graphs,
    _build_neighbour_index,
    fast_colour_refine,
    Partition,
    get_colour_signature,
)
from permutation import Permutation, group_order, orbit_and_transversal
from preprocessing import (
    is_tree, is_forest, is_connected, find_components, tree_canonical_label,
    tree_automorphisms, reduce_twins,
)
from math import factorial


# ---------------------------------------------------------------------------
# Graph copy utility
# ---------------------------------------------------------------------------

def _copy_graph(graph: Graph) -> Tuple[Graph, Dict[Vertex, Vertex]]:
    """Create an independent copy of *graph* with fresh vertex objects.

    Returns:
        (copy, mapping) where mapping maps each original vertex to its
        copy counterpart (by position in the vertex list).
    """
    copy = Graph(directed=graph.directed, n=len(graph))
    copy_verts = list(copy.vertices)
    orig_verts = list(graph.vertices)
    index_of = {v: i for i, v in enumerate(orig_verts)}
    vertex_map = {orig_verts[i]: copy_verts[i] for i in range(len(orig_verts))}

    seen_edges: set = set()
    for e in graph.edges:
        ti, hi = index_of[e.tail], index_of[e.head]
        edge_key = (min(ti, hi), max(ti, hi))
        if edge_key not in seen_edges:
            seen_edges.add(edge_key)
            copy.add_edge(Edge(copy_verts[ti], copy_verts[hi]))

    return copy, vertex_map
```

```python


# ---------------------------------------------------------------------------
# Branching helpers
# ---------------------------------------------------------------------------

class _BranchingContext:
    """Pre-computed state shared across all recursive calls for a pair
    of graphs, avoiding repeated allocation."""

    __slots__ = ("adj", "all_vertices", "verts_g", "verts_h",
                 "n_g", "base_colouring")

    def __init__(self, graph_g: Graph, graph_h: Graph, adj: Dict):
        self.adj = adj
        self.verts_g = list(graph_g)
        self.verts_h = list(graph_h)
        self.n_g = len(self.verts_g)
        self.all_vertices = self.verts_g + self.verts_h
        # Use degree as initial colour -- gives refinement a head start
        # by separating vertices of different degree before refinement.
        self.base_colouring = {v: v.degree for v in self.all_vertices}

    def refine_and_check(self, D: list, I: list):
        """Run colour refinement with α(D,I) and classify the result.

        Returns (partition, status) where status is 0 (unbalanced),
        1 (bijection), or 2 (branch needed).
        """
        # Build colouring: modify base, refine, then restore.
        # Individualized pairs get unique negative colours to avoid
        # collision with degree-based colours (which are >= 0).
        base = self.base_colouring
        depth = len(D)
        saved = []
        for i in range(depth):
            c = -(i + 1)
            saved.append((base[D[i]], base[I[i]]))
            base[D[i]] = c
            base[I[i]] = c

        partition = fast_colour_refine(self.all_vertices, self.adj, base)

        # Restore base colouring to degree values
        for i in range(depth):
            base[D[i]] = saved[i][0]
            base[I[i]] = saved[i][1]

        if not is_balanced_pair(self.verts_g, self.verts_h, partition):
            return None, 0
        if is_discrete_pair(self.verts_g, self.verts_h, partition):
            return partition, 1
        return partition, 2


def is_balanced_pair(verts_g: list, verts_h: list, partition: Partition) -> bool:
    """Check whether graphs G and H have identical colour distributions.

    Optimized for exactly two graphs: counts colours in G, then
    decrements for each colour in H.  Any negative count means
    unbalanced.  Faster than the general ``is_balanced`` in
    ``fast_colorref.py`` because it avoids building full signatures.

    Args:
        verts_g:    Vertex list of graph G.
        verts_h:    Vertex list of graph H.
        partition:  A stable Partition.

    Returns:
        True if the colour distribution is identical for both graphs.
    """
    vc = partition.vertex_colour
```

```python
    counts_g: Dict[int, int] = {}
    for v in verts_g:
        c = vc[v]
        counts_g[c] = counts_g.get(c, 0) + 1
    for v in verts_h:
        c = vc[v]
        cnt = counts_g.get(c, 0) - 1
        if cnt < 0:
            return False
        counts_g[c] = cnt
    return all(c == 0 for c in counts_g.values())


def is_discrete_pair(verts_g: list, verts_h: list, partition: Partition) -> bool:
    """Check if the colouring is discrete for both graphs.

    A discrete colouring assigns a unique colour to every vertex.
    Since the partition is assumed balanced, checking one graph is
    sufficient.  Returns False immediately on the first duplicate.

    Args:
        verts_g:    Vertex list of graph G.
        verts_h:    Vertex list of graph H.
        partition:  A stable Partition.

    Returns:
        True if every vertex in each graph has a distinct colour.
    """
    vc = partition.vertex_colour
    n = len(verts_g)
    if n != len(verts_h):
        return False
    # A discrete colouring has n distinct colours per graph
    # Since balanced, just check one graph
    seen = set()
    for v in verts_g:
        c = vc[v]
        if c in seen:
            return False
        seen.add(c)
    return True


def _choose_branching_class(verts_g: list, verts_h: list,
                            partition: Partition
                            ) -> Tuple[List[Vertex], List[Vertex]]:
    """Choose the smallest non-trivial colour class to branch on.

    A colour class is non-trivial if it contains >= 2 vertices in
    both graphs.  Picking the smallest such class minimises the
    branching factor at each recursion level, reducing the search
    tree size (Section 3.5 "Improvements").

    Args:
        verts_g:    Vertex list of graph G.
        verts_h:    Vertex list of graph H.
        partition:  A balanced, non-discrete Partition.

    Returns:
        ``(class_g, class_h)`` -- vertex lists for the chosen colour
        class in G and H respectively.
    """
    vc = partition.vertex_colour
    colours_g: Dict[int, list] = {}
    colours_h: Dict[int, list] = {}

    for v in verts_g:
        c = vc[v]
        if c in colours_g:
            colours_g[c].append(v)
        else:
            colours_g[c] = [v]
```

```
    for v in verts_h:
        c = vc[v]
        if c in colours_h:
            colours_h[c].append(v)
        else:
            colours_h[c] = [v]

    best_colour = -1
    best_size = 0x7FFFFFFF

    for colour, vg in colours_g.items():
        lg = len(vg)
        if lg < 2:
            continue
        vh = colours_h.get(colour)
        if vh is None or len(vh) < 2:
            continue
        if lg < best_size:
            best_size = lg
            best_colour = colour

    # Sort by degree descending so that cg[0] is the most constrained
    # vertex -- its high degree propagates more information through
    # colour refinement, leading to faster pruning.
    cg = sorted(colours_g[best_colour], key=lambda v: v.degree, reverse=True)
    ch = sorted(colours_h[best_colour], key=lambda v: v.degree, reverse=True)
    return cg, ch


def _extract_bijection(verts_g: list, verts_h: list,
                       partition: Partition) -> Dict[Vertex, Vertex]:
    """Extract the vertex mapping f: V(G) → V(H) from a discrete partition.

    When the partition is discrete and balanced, every colour appears
    exactly once in each graph.  This function matches each vertex in G
    to the vertex in H with the same colour, yielding the unique
    isomorphism defined by the partition.

    Args:
        verts_g:    Vertex list of graph G.
        verts_h:    Vertex list of graph H.
        partition:  A discrete, balanced Partition.

    Returns:
        Dict mapping each vertex of G to its image in H.
    """
    vc = partition.vertex_colour
    colour_to_h: Dict[int, Vertex] = {}
    for v in verts_h:
        colour_to_h[vc[v]] = v
    return {v: colour_to_h[vc[v]] for v in verts_g}


# ---------------------------------------------------------------------------
# GI branching  (Algorithm 2 -- CountIsomorphism)
# ---------------------------------------------------------------------------

_ORBIT_PRUNE_MAX_VERTICES = 100  # Only compute Aut(H) for small graphs


def _compute_h_aut_generators(graph_h: Graph, adj: Dict) -> Optional[List[Permutation]]:
    """Compute Aut(H) generators for orbit pruning in GI branching.

    Returns a list of generating permutations for Aut(H), or None if
    Aut(H) is trivial or the graph is too large for the overhead to
    be worthwhile.
    """
    n = len(graph_h)
    if n > _ORBIT_PRUNE_MAX_VERTICES:
        return None  # Aut computation too expensive for large graphs
    if is_forest(graph_h):
        return None  # trees are handled separately
```

```python
    local_adj = _build_neighbour_index(list(graph_h))
    result = _count_aut_core(graph_h, local_adj, None, return_generators=True)
    if isinstance(result, int):
        return None  # trivial group
    order, generators = result
    if order <= 1 or not generators:
        return None
    return generators


def count_isomorphisms(graph_g: Graph,
                       graph_h: Graph,
                       D: List[Vertex],
                       I: List[Vertex],
                       adj: Dict,
                       gi_only: bool = False,
                       h_aut_generators: Optional[List[Permutation]] = None) -> int:
    """Count isomorphisms from *graph_g* to *graph_h*.

    Implements Algorithm 2 (CountIsomorphism) from Chapter 3.
    Starting from the individualization sequences D (vertices in G) and
    I (their images in H), recursively branches on colour classes until
    the partition is either unbalanced (no isomorphism) or discrete
    (exactly one isomorphism).

    Args:
        graph_g:           Source graph.
        graph_h:           Target graph.
        D:                 Initial individualization sequence in G (usually []).
        I:                 Initial individualization sequence in H (usually []).
        adj:               Pre-computed adjacency dict for all vertices in G and H.
        gi_only:           If True, short-circuit after finding the first
                           isomorphism (returns 1 instead of counting all).
        h_aut_generators:  Optional Aut(H) generators.  If provided, orbit
                           pruning reduces the branching factor by only trying
                           one candidate per orbit at each level.

    Returns:
        Number of isomorphisms (or 1 if gi_only and at least one exists).
    """
    ctx = _BranchingContext(graph_g, graph_h, adj)

    def _count(D: list, I: list,
               aut_gens: Optional[List[Permutation]]) -> int:
        """Recursive core: refine, check, and branch if needed."""
        partition, status = ctx.refine_and_check(D, I)
        if status == 0:
            return 0
        if status == 1:
            return 1

        cg, ch = _choose_branching_class(ctx.verts_g, ctx.verts_h, partition)
        x = cg[0]

        # Orbit pruning: compute orbits of Aut(H) stabilizer within
        # the current colour class to reduce branching factor.
        # At each level, only generators that fix all previously
        # individualized H-vertices are valid (stabilizer subgroup).
        candidates = ch
        if aut_gens:
            # Compute orbits within ch using current generators
            ch_set = set(ch)
            seen: set = set()
            pruned = []
            for y in ch:
                if y in seen:
                    continue
                pruned.append(y)
                orb, _ = orbit_and_transversal(aut_gens, y)
                seen |= (orb & ch_set)
            candidates = pruned

```

```
        num = 0
        D.append(x)
        for y in candidates:
            I.append(y)
            # Compute stabilizer generators for next level:
            # keep only generators that fix y.
            next_gens = None
            if aut_gens:
                next_gens = [g for g in aut_gens if g(y) == y]
                if not next_gens:
                    next_gens = None
            num += _count(D, I, next_gens)
            I.pop()
            if gi_only and num > 0:
                D.pop()
                return num
        D.pop()
        return num

    return _count(list(D), list(I), h_aut_generators)


# ---------------------------------------------------------------------------
# Preprocessing: forest automorphisms
# ---------------------------------------------------------------------------

def _forest_automorphisms(graph: Graph) -> int:
    """Count |Aut(F)| for a forest F using AHU (O(n)).

    Each component's automorphisms are computed independently, then
    multiplied together.  Isomorphic components can be permuted,
    contributing an additional k! factor per group of k isomorphic trees.
    """
    components = find_components(graph)

    # Group components by their canonical tree label
    label_groups: Dict[Tuple, List[List[Vertex]]] = defaultdict(list)
    for comp in components:
        label = tree_canonical_label(comp)
        label_groups[label].append(comp)

    total = 1
    for label, comps in label_groups.items():
        for comp in comps:
            total *= tree_automorphisms(comp)
        # k isomorphic components can be permuted: ×k!
        total *= factorial(len(comps))

    return total


# ---------------------------------------------------------------------------
# #Aut branching  (Algorithm 9 -- UpdateGeneratingSet + pruning)
# ---------------------------------------------------------------------------

def _component_graph(vertices: List[Vertex], parent: Graph) -> Graph:
    """Build a new Graph induced by *vertices* from *parent*.

    Creates a fresh Graph with new Vertex/Edge objects containing
    only the vertices in the given list and edges between them.
    Used to isolate connected components for independent solving.

    Args:
        vertices:  List of vertices forming the induced subgraph.
        parent:    The original graph containing these vertices.

    Returns:
        A new Graph object representing the induced subgraph.
    """
    vset = set(vertices)
    sub = Graph(directed=parent.directed, n=len(vertices))
    sub_verts = list(sub.vertices)
```

```
    old_to_new = {vertices[i]: sub_verts[i] for i in range(len(vertices))}

    seen: set = set()
    for v in vertices:
        for e in v.incidence:
            other = e.other_end(v)
            if other in vset:
                ek = (min(id(e.tail), id(e.head)),
                      max(id(e.tail), id(e.head)))
                if ek not in seen:
                    seen.add(ek)
                    sub.add_edge(Edge(old_to_new[e.tail], old_to_new[e.head]))
    return sub


def _disconnected_automorphisms(graph: Graph, adj: Dict) -> int:
    """Count |Aut(G)| for a disconnected non-forest graph.

    Decomposes G into connected components and solves each
    independently.  The total automorphism count is the product of
    per-component counts, times k! for each group of k mutually
    isomorphic components (since isomorphic components can be freely
    permuted among themselves).

    Component isomorphism is checked via vertex/edge count filtering
    followed by exact branching.

    Args:
        graph:  A disconnected, non-forest graph.
        adj:    Pre-computed adjacency dict.

    Returns:
        |Aut(graph)|
    """
    components = find_components(graph)
    if len(components) == 1:
        return _count_aut_core(graph, adj, None)

    # Build a subgraph for each component and solve independently
    comp_graphs: List[Graph] = []
    comp_auts: List[int] = []
    for comp in components:
        sub = _component_graph(comp, graph)
        sub_adj = _build_neighbour_index(list(sub))
        if is_forest(sub):
            aut = _forest_automorphisms(sub)
        else:
            aut = _count_aut_core(sub, sub_adj, None)
        comp_graphs.append(sub)
        comp_auts.append(aut)

    # Group isomorphic components for the k! permutation factor.
    # Use colour refinement signatures as a cheap first grouping,
    # then exact isomorphism within signature groups.
    assigned = [False] * len(comp_graphs)
    total = 1

    for i in range(len(comp_graphs)):
        if assigned[i]:
            continue
        assigned[i] = True
        group_count = 1
        total *= comp_auts[i]
        for j in range(i + 1, len(comp_graphs)):
            if assigned[j]:
                continue
            if len(comp_graphs[i]) != len(comp_graphs[j]):
                continue
            if len(comp_graphs[i].edges) != len(comp_graphs[j].edges):
                continue
            pair_adj = _build_neighbour_index(
                list(comp_graphs[i]) + list(comp_graphs[j])
```

```
            )
            if count_isomorphisms(comp_graphs[i], comp_graphs[j],
                                  [], [], pair_adj, gi_only=True) > 0:
                assigned[j] = True
                total *= comp_auts[j]
                group_count += 1
        total *= factorial(group_count)

    return total


def count_automorphisms(graph: Graph,
                        adj: Dict[Vertex, List[Vertex]]) -> int:
    """Count |Aut(graph)| using generating sets and pruning.

    Preprocessing (Chapter 6):
      - Trees/forests are solved in O(n) via AHU.
      - Disconnected graphs are decomposed into components.

    General case (Chapters 3+5):
      1. Build a generating set X of Aut(G) by comparing G to a copy H.
      2. Prune branches using Lemma 5.11.
      3. Compute |Aut(G)| = |⟨X⟩| via the orbit-stabiliser theorem.

    Args:
        graph:  The graph.
        adj:    Pre-computed adjacency lists.

    Returns:
        |Aut(graph)|
    """
    # Preprocessing: forests are solved directly via AHU
    if is_forest(graph):
        return _forest_automorphisms(graph)

    # Disconnected non-forest: decompose into components
    if not is_connected(graph):
        return _disconnected_automorphisms(graph, adj)

    # Twin reduction: collapse twin groups and multiply by k! factors
    reduced, aut_factor, pre_colouring = reduce_twins(graph)
    if len(reduced) < len(graph):
        reduced_adj = _build_neighbour_index(list(reduced))
        return aut_factor * _count_aut_core(reduced, reduced_adj, pre_colouring)

    return _count_aut_core(graph, adj, None)



def _count_aut_core(graph: Graph, adj: Dict,
                    pre_colouring: Optional[Dict],
                    return_generators: bool = False):
    """Core #Aut via generating sets + Lemma 5.11 pruning.

    Args:
        graph:              The graph (possibly twin-reduced).
        adj:                Pre-computed adjacency lists.
        pre_colouring:      Optional initial colouring from twin reduction.
        return_generators:  If True, return (order, generators) instead of order.
    """
    graph_copy, vertex_map = _copy_graph(graph)

    for v in graph_copy:
        adj[v] = v.neighbours

    orig_verts = list(graph.vertices)
    copy_verts = list(graph_copy.vertices)
    copy_to_orig = {copy_verts[i]: orig_verts[i]
                    for i in range(len(orig_verts))}

    ctx = _BranchingContext(graph, graph_copy, adj)

```

```python
    # Apply pre-colouring if provided (from twin reduction)
    if pre_colouring is not None:
        for v in orig_verts:
            ctx.base_colouring[v] = pre_colouring.get(v, 0)
        for i, v in enumerate(copy_verts):
            ctx.base_colouring[v] = pre_colouring.get(orig_verts[i], 0)

    generating_set: List[Permutation] = []

    def _to_permutation(partition: Partition) -> Permutation:
        """Convert a discrete partition into an automorphism permutation.

        Extracts the G→H bijection from the partition, then maps it
        back to G's own vertices (since H is a copy of G) to produce
        a self-isomorphism (automorphism).
        """
        bijection = _extract_bijection(ctx.verts_g, ctx.verts_h, partition)
        return Permutation({v: copy_to_orig[bijection[v]] for v in orig_verts})

    D: list = []
    I: list = []

    def _update() -> None:
        """Algorithm 9 -- build generating set with Lemma 5.11 pruning.

        Trivial branch (x → x'): recurse with _update to find ALL
        stabiliser generators.  Non-trivial branches (x → y): call
        _find_one to find just ONE automorphism per coset -- combined
        with the stabiliser, this suffices to generate all automorphisms
        mapping x to y.
        """
        partition, status = ctx.refine_and_check(D, I)
        if status == 0:
            return
        if status == 1:
            perm = _to_permutation(partition)
            if not perm.is_identity():
                generating_set.append(perm)
            return

        cg, ch = _choose_branching_class(ctx.verts_g, ctx.verts_h, partition)
        x = cg[0]
        x_copy = vertex_map[x]
        others = [y for y in ch if y is not x_copy]

        D.append(x)
        if x_copy in ch:
            I.append(x_copy)
            _update()
            I.pop()

        for y in others:
            I.append(y)
            _find_one(cg, ch)
            I.pop()
        D.pop()

    def _find_one(parent_cg, parent_ch) -> bool:
        """Find exactly one non-identity automorphism in this branch.

        Returns True (and appends to generating_set) as soon as a
        single automorphism is found.  Returns False if the branch
        is exhausted without finding one.  This is the Lemma 5.11
        optimization: one coset representative per non-trivial branch.
        """
        partition, status = ctx.refine_and_check(D, I)
        if status == 0:
            return False
        if status == 1:
            perm = _to_permutation(partition)
            if perm.is_identity():
                return False
```

```
            generating_set.append(perm)
            return True

        cg, ch = _choose_branching_class(ctx.verts_g, ctx.verts_h, partition)
        x = cg[0]
        D.append(x)
        for y in ch:
            I.append(y)
            found = _find_one(cg, ch)
            I.pop()
            if found:
                D.pop()
                return True
        D.pop()
        return False

    _update()

    # Clean up copy-graph entries from the shared adj dict
    for v in copy_verts:
        adj.pop(v, None)

    if return_generators:
        return group_order(generating_set, orig_verts), generating_set
    return group_order(generating_set, orig_verts)


# ---------------------------------------------------------------------------
# GI problem -- find equivalence classes
# ---------------------------------------------------------------------------

def _tree_equivalence_classes(graphs: List[Graph]) -> List[List[int]]:
    """Partition trees into equivalence classes using AHU canonical labels.

    O(n) per tree -- no branching needed.
    """
    label_groups: Dict[Tuple, List[int]] = defaultdict(list)
    for i, G in enumerate(graphs):
        label = tree_canonical_label(list(G.vertices))
        label_groups[label].append(i)
    return sorted([sorted(g) for g in label_groups.values()],
                  key=lambda c: c[0])


def _are_isomorphic(graph_g: Graph, graph_h: Graph, adj: Dict,
                    h_aut_gens: Optional[List[Permutation]] = None) -> bool:
    """Decide whether *graph_g* and *graph_h* are isomorphic.

    Applies cheap early-exit checks (vertex count, edge count) before
    running the expensive branching algorithm with ``gi_only=True``.
    When *h_aut_gens* is provided, orbit pruning reduces the branching
    factor by only trying one candidate per Aut(H) orbit at each level.

    Args:
        graph_g:     First graph.
        graph_h:     Second graph.
        adj:         Pre-computed adjacency dict for vertices of both graphs.
        h_aut_gens:  Optional Aut(H) generators for orbit pruning.

    Returns:
        True if the two graphs are isomorphic.
    """
    if len(graph_g) != len(graph_h):
        return False
    if len(graph_g.edges) != len(graph_h.edges):
        return False
    return count_isomorphisms(graph_g, graph_h, [], [], adj,
                              gi_only=True,
                              h_aut_generators=h_aut_gens) > 0


def find_equivalence_classes(graphs: List[Graph],
```

```
                                adj: Dict[Vertex, List[Vertex]]
                                ) -> List[List[int]]:
    """Partition graphs into equivalence classes of isomorphic graphs.

    Uses fast colour refinement as a first pass to cheaply separate
    obviously non-isomorphic graphs, then applies branching only within
    groups that colour refinement cannot distinguish.

    Args:
        graphs:  List of Graph objects.
        adj:     Pre-computed adjacency lists for all vertices.

    Returns:
        A sorted list of sorted graph-index lists.
    """
    partition = colour_refine_graphs(graphs)
    sig_groups: Dict[Tuple, List[int]] = defaultdict(list)
    for i, G in enumerate(graphs):
        sig = get_colour_signature(G, partition)
        sig_groups[sig].append(i)

    equivalence_classes: List[List[int]] = []
    for indices in sig_groups.values():
        if len(indices) == 1:
            equivalence_classes.append(indices)
            continue

        remaining = set(indices)
        # Pre-compute Aut generators for orbit pruning.
        # Cache per graph to reuse across multiple pair tests.
        aut_cache: Dict[int, Optional[List[Permutation]]] = {}
        for i in indices:
            if i not in remaining:
                continue
            eq_class = [i]
            remaining.remove(i)
            to_remove = []
            for j in remaining:
                # Compute Aut(H) generators for graph j on first use
                if j not in aut_cache:
                    aut_cache[j] = _compute_h_aut_generators(graphs[j], adj)
                if _are_isomorphic(graphs[i], graphs[j], adj,
                                   h_aut_gens=aut_cache[j]):
                    eq_class.append(j)
                    to_remove.append(j)
            for j in to_remove:
                remaining.remove(j)
            equivalence_classes.append(sorted(eq_class))

    return sorted(equivalence_classes, key=lambda c: c[0])


# ---------------------------------------------------------------------------
# Public API -- solve from file
# ---------------------------------------------------------------------------

def solve(filename: str) -> None:
    """Solve the GI and/or #Aut problem for the given file and print results.

    Automatically detects the problem type from the filename:
        - ``*GI.grl``     → equivalence classes only
        - ``*Aut.grl/gr`` → automorphism counts only
        - ``*GIAut.grl``  → both

    Args:
        filename:  Path to a ``.grl`` or ``.gr`` file.
    """
    name = filename.rsplit("/", 1)[-1].rsplit(".", 1)[0]
    do_gi = "GI" in name
    do_aut = "Aut" in name

    if not do_gi and not do_aut:
```

```python
        if filename.endswith(".gr"):
            do_aut = True
        else:
            do_gi = True

    is_single = filename.endswith(".gr")

    if is_single:
        with open(filename, "r") as f:
            graph = load_graph(f, graph_class=Graph, read_list=False)
        adj = _build_neighbour_index(list(graph))
        aut = count_automorphisms(graph, adj)
        print(f"Number of automorphisms: {aut}")
        return

    with open(filename, "r") as f:
        graphs = load_graph(f, graph_class=Graph, read_list=True)

    all_verts = [v for G in graphs for v in G]
    adj = _build_neighbour_index(all_verts)

    # Preprocessing: if ALL graphs are trees, use AHU for both GI and #Aut
    all_trees = all(is_tree(G) for G in graphs)

    if do_gi and not do_aut:
        if all_trees:
            classes = _tree_equivalence_classes(graphs)
        else:
            classes = find_equivalence_classes(graphs, adj)
        print("Sets of isomorphic graphs:")
        for eq_class in classes:
            print(eq_class)

    elif do_aut and not do_gi:
        print("Number of automorphisms for each graph:")
        for i, G in enumerate(graphs):
            local_adj = _build_neighbour_index(list(G))
            aut = count_automorphisms(G, local_adj)
            print(f"{i}: {aut}")

    else:
        if all_trees:
            classes = _tree_equivalence_classes(graphs)
        else:
            classes = find_equivalence_classes(graphs, adj)
        print("Sets of isomorphic graphs with automorphisms:")
        for eq_class in classes:
            rep = eq_class[0]
            local_adj = _build_neighbour_index(list(graphs[rep]))
            aut = count_automorphisms(graphs[rep], local_adj)
            print(f"{eq_class}: {aut}")


# ---------------------------------------------------------------------------
# Entry point
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    import sys
    if len(sys.argv) < 2:
        print("Usage: python branching.py <graph_file.grl|.gr>")
        sys.exit(1)
    solve(sys.argv[1])
```

Listing 5: *branching.py — Branching for GI and #Aut*

F. Source Code: permutation.py

Permutation group utilities: orbit/transversal computation (Algorithm 10), Schreier-Sims stabiliser chains, group order via orbit-stabiliser (Proposition 5.18), and membership testing via sifting (Proposition 5.20).

```python
"""
Permutation group utilities for automorphism counting.

Provides a lightweight Permutation class and the algebraic machinery
needed to avoid enumerating every automorphism:

  * Orbit and transversal computation   (Algorithm 10, Section 5.5.4)
  * Stabiliser chain via Schreier-Sims  (Theorem 5.19)
  * Group order via orbit-stabiliser    (Proposition 5.18)
  * Membership testing via sifting       (Proposition 5.20)

All permutations act on abstract elements (vertex objects).  Composition
follows the reader's convention: (p * q)(x) = p(q(x)).
"""

from __future__ import annotations
from collections import deque
from typing import Dict, List, Optional, Set, Tuple


# ---------------------------------------------------------------------------
# Permutation
# ---------------------------------------------------------------------------

class Permutation:
    """A bijection on a finite set, stored as a dictionary.

    Only non-fixed-point mappings are stored, so the identity permutation
    is represented by an empty dict.
    """

    __slots__ = ("_map", "_hash")

    def __init__(self, mapping: Optional[Dict] = None):
        """Create a permutation from an explicit mapping dict.

        Only non-fixed-point entries are stored: if mapping[k] == k,
        that entry is dropped.  Passing ``None`` or ``{}`` gives the
        identity permutation.

        Args:
            mapping:  Dict of element → image.  ``None`` for identity.
        """
        if mapping is None:
            self._map: Dict = {}
        else:
            self._map = {k: v for k, v in mapping.items() if k != v}
        self._hash: Optional[int] = None

    def __call__(self, x):
        """Apply the permutation: return σ(x)."""
        return self._map.get(x, x)

    def __mul__(self, other: Permutation) -> Permutation:
        """Compose: (self * other)(x) = self(other(x))."""
        keys = set(self._map) | set(other._map)
        return Permutation({k: self(other(k)) for k in keys})

    def inverse(self) -> Permutation:
        """Return σ⁻¹."""
        return Permutation({v: k for k, v in self._map.items()})

    def is_identity(self) -> bool:
        return len(self._map) == 0
```

```python

    @property
    def support(self) -> Set:
        """Set of elements moved by this permutation."""
        return set(self._map.keys())

    def __eq__(self, other):
        if not isinstance(other, Permutation):
            return NotImplemented
        return self._map == other._map

    def __hash__(self):
        if self._hash is None:
            self._hash = hash(frozenset(self._map.items()))
        return self._hash

    def __repr__(self):
        if self.is_identity():
            return "Permutation()"
        cycles = self._cycle_notation()
        return "".join(f"({' '.join(str(x) for x in c)})" for c in cycles)

    def _cycle_notation(self) -> List[List]:
        """Return the permutation in cycle notation as a list of cycles.

        Each cycle is a list of elements.  Fixed points are omitted.
        Cycles are sorted by their smallest element for deterministic
        output.
        """
        visited: Set = set()
        cycles: List[List] = []
        for k in sorted(self._map, key=str):
            if k in visited:
                continue
            cycle = []
            x = k
            while x not in visited:
                visited.add(x)
                cycle.append(x)
                x = self(x)
            if len(cycle) > 1:
                cycles.append(cycle)
        return cycles


IDENTITY = Permutation()


# ---------------------------------------------------------------------------
# Orbit and transversal  (Algorithm 10)
# ---------------------------------------------------------------------------

def orbit_and_transversal(generators: List[Permutation],
                          alpha) -> Tuple[Set, Dict]:
    """Compute the orbit of *alpha* under ⟨generators⟩ and an orbit
    transversal.

    Returns:
        (orbit, transversal) where transversal[β] is a permutation
        mapping alpha to β.
    """
    orbit: Set = {alpha}
    transversal: Dict = {alpha: IDENTITY}
    queue: deque = deque([alpha])

    while queue:
        beta = queue.popleft()
        for g in generators:
            gamma = g(beta)
            if gamma not in orbit:
                orbit.add(gamma)
                transversal[gamma] = g * transversal[beta]
```

```
                queue.append(gamma)

    return orbit, transversal


# ---------------------------------------------------------------------------
# Stabiliser chain  (Schreier-Sims)
# ---------------------------------------------------------------------------

class _StabiliserLevel:
    """One level of a stabiliser chain.

    Stores the base point, orbit, transversal and generators for the
    stabiliser subgroup at this level.
    """
    __slots__ = ("alpha", "orbit", "transversal", "generators")

    def __init__(self, alpha, generators: List[Permutation]):
        """Build one level of the stabiliser chain.

        Computes the orbit and transversal of *alpha* under
        *generators* immediately on construction.

        Args:
            alpha:       The base point for this level.
            generators:  Generating set for the group at this level.
        """
        self.alpha = alpha
        self.generators = list(generators)
        self.orbit, self.transversal = orbit_and_transversal(
            self.generators, alpha
        )


def _build_chain(generators: List[Permutation],
                 base: List) -> List[_StabiliserLevel]:
    """Build a stabiliser chain for ⟨generators⟩ using the given base.

    Each level i represents G^(i) = Stab(base[0], ..., base[i-1]),
    with G^(0) = G.

    Uses Schreier generators with sifting to keep generator lists small.
    """
    chain: List[_StabiliserLevel] = []
    current_gens = list(generators)

    for alpha in base:
        if not current_gens:
            break

        level = _StabiliserLevel(alpha, current_gens)
        chain.append(level)

        # Compute Schreier generators for the stabiliser of alpha,
        # but sift them through deeper levels to avoid redundancy
        next_gens: List[Permutation] = []
        seen: Set[Permutation] = set()

        for beta in level.orbit:
            u_beta = level.transversal[beta]
            for g in level.generators:
                gamma = g(beta)
                u_gamma = level.transversal[gamma]
                schreier = u_gamma.inverse() * g * u_beta
                if not schreier.is_identity() and schreier not in seen:
                    seen.add(schreier)
                    next_gens.append(schreier)

        current_gens = next_gens

    return chain

```

```python

def _choose_base(generators: List[Permutation],
                 elements: List) -> List:
    """Choose a base -- a sequence of elements whose pointwise stabiliser
    is trivial.  We pick elements that are moved by the generators."""
    base = []
    moved = set()
    for g in generators:
        moved |= g.support

    # Order by elements list, taking only moved elements
    for e in elements:
        if e in moved:
            base.append(e)

    return base


# ---------------------------------------------------------------------------
# Group order  (Proposition 5.18)
# ---------------------------------------------------------------------------

def group_order(generators: List[Permutation],
                elements: List) -> int:
    """Compute |⟨generators⟩| using a stabiliser chain.

    The order is the product of orbit sizes at each level of the chain:
    |G| = |orbit_1| × |orbit_2| × ... × |orbit_k|

    Args:
        generators:  Generating set of the group.
        elements:    The elements the group acts on (vertices).

    Returns:
        The order of the group.
    """
    if not generators:
        return 1

    base = _choose_base(generators, elements)
    if not base:
        return 1

    chain = _build_chain(generators, base)

    order = 1
    for level in chain:
        order *= len(level.orbit)

    return order


# ---------------------------------------------------------------------------
# Membership testing  (Proposition 5.20 -- Sifting)
# ---------------------------------------------------------------------------

def is_member(perm: Permutation,
              generators: List[Permutation],
              elements: List) -> bool:
    """Test whether *perm* ∈ ⟨generators⟩ by sifting through a
    stabiliser chain.

    Args:
        perm:        The permutation to test.
        generators:  Generating set of the group.
        elements:    Elements the group acts on.

    Returns:
        True iff *perm* belongs to the group.
    """
    if perm.is_identity():
        return True
```

```
    if not generators:
        return False

    base = _choose_base(generators, elements)
    chain = _build_chain(generators, base)

    current = perm
    for level in chain:
        beta = current(level.alpha)
        if beta not in level.orbit:
            return False
        u_beta = level.transversal[beta]
        current = u_beta.inverse() * current
        if current.is_identity():
            return True

    return current.is_identity()
```

Listing 6: *permutation.py — Permutation group utilities*

G. Source Code: preprocessing.py

Preprocessing: tree/forest detection, AHU canonical labelling (Section 6.2), twin detection and iterative reduction (Section 6.3), and connected component decomposition (Section 6.4).

```
"""
Preprocessing -- structural detection and simplification (Chapter 6).

Before running the full branching algorithm, we can detect special graph
structures and handle them more efficiently:

  * **Trees / Forests**:  Isomorphism and #Aut solved in O(n) via the
    AHU algorithm (Section 6.2).
  * **Twins / False twins**:  Sets of (false) twins are collapsed; the
    automorphism count is multiplied by k! per set of k twins (Section 6.3).
  * **Connected components**:  Decompose disconnected graphs and solve
    per-component (Section 6.4).
"""

from collections import defaultdict, deque
from math import factorial
from typing import Dict, List, Optional, Set, Tuple
from graph import Graph, Vertex


# ---------------------------------------------------------------------------
# Component decomposition  (Section 6.4)
# ---------------------------------------------------------------------------

def find_components(graph: Graph) -> List[List[Vertex]]:
    """Find connected components via BFS.

    Args:
        graph:  The graph.

    Returns:
        A list of components, each a list of vertices.
    """
    visited: Set[Vertex] = set()
    components: List[List[Vertex]] = []

    for start in graph:
        if start in visited:
            continue
        component: List[Vertex] = []
        queue: deque = deque([start])
        visited.add(start)
        while queue:
```

```
            v = queue.popleft()
            component.append(v)
            for u in v.neighbours:
                if u not in visited:
                    visited.add(u)
                    queue.append(u)
        components.append(component)

    return components


def is_connected(graph: Graph) -> bool:
    """Check whether *graph* is connected."""
    if len(graph) == 0:
        return True
    components = find_components(graph)
    return len(components) == 1


# ---------------------------------------------------------------------------
# Tree / Forest detection  (Section 6.2)
# ---------------------------------------------------------------------------

def is_tree(graph: Graph) -> bool:
    """Check whether *graph* is a tree (connected acyclic graph)."""
    n = len(graph)
    if n == 0:
        return True
    return is_connected(graph) and len(graph.edges) == n - 1


def is_forest(graph: Graph) -> bool:
    """Check whether *graph* is a forest (acyclic graph, possibly
    disconnected)."""
    components = find_components(graph)
    n = len(graph)
    m = len(graph.edges)
    return m == n - len(components)


# ---------------------------------------------------------------------------
# AHU algorithm for rooted tree isomorphism  (Section 6.2)
# ---------------------------------------------------------------------------

def _find_center(vertices: List[Vertex]) -> List[Vertex]:
    """Find the center of a tree (1 or 2 vertices).

    Iteratively removes leaves until 1 or 2 vertices remain.
    """
    if len(vertices) <= 2:
        return list(vertices)

    vertex_set = set(vertices)
    degree: Dict[Vertex, int] = {}
    for v in vertex_set:
        degree[v] = sum(1 for u in v.neighbours if u in vertex_set)

    remaining = set(vertex_set)
    leaves = [v for v in remaining if degree[v] <= 1]

    while len(remaining) > 2:
        new_leaves: List[Vertex] = []
        for leaf in leaves:
            remaining.discard(leaf)
            for u in leaf.neighbours:
                if u in remaining:
                    degree[u] -= 1
                    if degree[u] == 1:
                        new_leaves.append(u)
        leaves = new_leaves

    return list(remaining)
```

```python


def ahu_label(root: Vertex, parent: Optional[Vertex],
              vertex_set: Set[Vertex]) -> Tuple:
    """Compute the AHU canonical label for a rooted subtree.

    The label is a recursively-defined sorted tuple of children's labels.
    Leaves get label ``()``.

    Args:
        root:        The root of the subtree.
        parent:      The parent of *root* (None for the actual root).
        vertex_set:  The set of vertices belonging to this tree component.

    Returns:
        A hashable canonical label (nested tuples).
    """
    children_labels = []
    for child in root.neighbours:
        if child != parent and child in vertex_set:
            children_labels.append(ahu_label(child, root, vertex_set))
    return tuple(sorted(children_labels))


def tree_canonical_label(vertices: List[Vertex]) -> Tuple:
    """Compute a canonical label for an unrooted tree.

    Roots at the center (1 or 2 vertices).  If there are 2 centers,
    we use both and pick the lexicographically smallest.

    Args:
        vertices:  The vertices of a single tree component.

    Returns:
        A canonical label (hashable) that is identical for isomorphic trees.
    """
    vertex_set = set(vertices)
    centers = _find_center(vertices)

    if len(centers) == 1:
        return ahu_label(centers[0], None, vertex_set)

    # Two centers -- root at each and take the canonical minimum
    label_a = ahu_label(centers[0], None, vertex_set)
    label_b = ahu_label(centers[1], None, vertex_set)
    return min(label_a, label_b)


def tree_automorphisms(vertices: List[Vertex]) -> int:
    """Count |Aut(T)| for a tree T using the AHU structure.

    For a rooted tree, |Aut(T)| = ∏_v (k_v! × ∏_{children c of v} |Aut(T_c)|)
    where k_v is the number of children of v with identical subtree labels
    (grouped by label, each group contributing group_size!).

    For an unrooted tree, we root at the center.  If there are two centers
    connected by an edge and their subtrees are isomorphic, we multiply by 2.
    """
    vertex_set = set(vertices)
    centers = _find_center(vertices)

    def _count_rooted(root: Vertex, parent: Optional[Vertex]) -> Tuple[int, Tuple]:
        """Returns (aut_count, canonical_label) for rooted subtree."""
        children_data: List[Tuple[int, Tuple]] = []
        for child in root.neighbours:
            if child != parent and child in vertex_set:
                children_data.append(_count_rooted(child, root))

        # Group children by their canonical label
        label_groups: Dict[Tuple, int] = defaultdict(int)
        aut = 1
        for child_aut, child_label in children_data:
```

```
            aut *= child_aut
            label_groups[child_label] += 1

        # For each group of k identical subtrees: multiply by k!
        for count in label_groups.values():
            aut *= factorial(count)

        label = tuple(sorted(child_label for _, child_label in children_data))
        return aut, label

    if len(centers) == 1:
        aut, _ = _count_rooted(centers[0], None)
        return aut

    # Two centers
    aut_a, label_a = _count_rooted(centers[0], centers[1])
    aut_b, label_b = _count_rooted(centers[1], centers[0])
    total = aut_a * aut_b

    # If the two halves are isomorphic, we can also swap them → ×2
    if label_a == label_b:
        total *= 2

    return total


# ---------------------------------------------------------------------------
# Twins detection  (Section 6.3)
# ---------------------------------------------------------------------------

def find_twin_groups(graph: Graph) -> Tuple[List[Set[Vertex]], List[Set[Vertex]]]:
    """Detect sets of true twins and false twins.

    True twins: u,v adjacent AND N(u)\\{v} = N(v)\\{u}.
    False twins: N(u) = N(v) (not adjacent to each other).

    Returns:
        (true_twin_groups, false_twin_groups) -- each a list of sets,
        where each set has ≥ 2 vertices.
    """
    # Signature for false twins: frozenset of neighbours
    # Signature for true twins: frozenset of neighbours ∪ {self}
    false_sig: Dict[frozenset, List[Vertex]] = defaultdict(list)
    true_sig: Dict[frozenset, List[Vertex]] = defaultdict(list)

    for v in graph:
        nbrs = frozenset(v.neighbours)
        false_sig[nbrs].append(v)
        true_sig[nbrs | {v}].append(v)

    false_groups = [set(g) for g in false_sig.values() if len(g) >= 2]
    true_groups = [set(g) for g in true_sig.values() if len(g) >= 2]

    return true_groups, false_groups


# ---------------------------------------------------------------------------
# Iterative twin elimination  (Section 6.3)
# ---------------------------------------------------------------------------

def reduce_twins(graph: Graph) -> Tuple[Graph, int, Dict]:
    """Iteratively detect and collapse (false) twin groups.

    For each group of k twins, we keep one representative vertex and
    give it a unique colour encoding the group size.  The automorphism
    count must be multiplied by k! per group.  After each round of
    elimination, new twin groups may appear, so we repeat until no
    more are found.

    Args:
        graph:  The input graph.

```

```
    Returns:
        (reduced_graph, aut_factor, initial_colouring)
        - reduced_graph:  A new Graph with twins collapsed.
        - aut_factor:     Multiplicative factor for |Aut| from eliminated twins.
        - initial_colouring:  Dict mapping each vertex of the reduced graph
                              to an initial colour that encodes twin-group
                              structure (for use in colour refinement).
    """
    from graph import Edge

    vertices = list(graph.vertices)
    n = len(vertices)

    # Build adjacency as sets of vertex indices for fast manipulation
    idx = {v: i for i, v in enumerate(vertices)}
    adj_sets: List[Set[int]] = [set() for _ in range(n)]
    for e in graph.edges:
        ti, hi = idx[e.tail], idx[e.head]
        adj_sets[ti].add(hi)
        adj_sets[hi].add(ti)

    alive = set(range(n))        # vertices still in the graph
    colour = [0] * n     # each vertex's colour (encodes twin history)
    next_colour = 1
    aut_factor = 1

    # Representative colour encodes (twin_type, group_size) so that:
    # - Representatives of different-type/size groups get distinct colours
    # - Representatives of same-type/size groups share a colour (allowing
    #   colour refinement to discover if they're structurally equivalent)
    # - Non-twin vertices keep colour 0
    type_size_to_colour: Dict[Tuple, int] = {}
    round_num = 0

    changed = True
    while changed:
        round_num += 1
        changed = False

        # Detect false twins: non-adjacent, identical neighbourhoods
        false_sig: Dict[Tuple, List[int]] = defaultdict(list)
        for i in alive:
            sig = (colour[i], frozenset(adj_sets[i] & alive))
            false_sig[sig].append(i)

        for sig, group in false_sig.items():
            if len(group) < 2:
                continue
            changed = True
            k = len(group)
            aut_factor *= factorial(k)

            ts_key = ("F", k, round_num)
            if ts_key not in type_size_to_colour:
                type_size_to_colour[ts_key] = next_colour
                next_colour += 1

            keep = group[0]
            colour[keep] = type_size_to_colour[ts_key]
            for rem in group[1:]:
                alive.discard(rem)
                for nb in adj_sets[rem]:
                    adj_sets[nb].discard(rem)

        # Detect true twins: adjacent, N(u) ∪ {u} identical
        true_sig: Dict[Tuple, List[int]] = defaultdict(list)
        for i in alive:
            sig = (colour[i], frozenset((adj_sets[i] & alive) | {i}))
            true_sig[sig].append(i)

        for sig, group in true_sig.items():
            if len(group) < 2:
```

```
                continue
            changed = True
            k = len(group)
            aut_factor *= factorial(k)

            ts_key = ("T", k, round_num)
            if ts_key not in type_size_to_colour:
                type_size_to_colour[ts_key] = next_colour
                next_colour += 1

            keep = group[0]
            colour[keep] = type_size_to_colour[ts_key]
            for rem in group[1:]:
                alive.discard(rem)
                for nb in adj_sets[rem]:
                    adj_sets[nb].discard(rem)

    # Build the reduced graph
    alive_list = sorted(alive)
    reduced = Graph(directed=graph.directed, n=len(alive_list))
    new_verts = list(reduced.vertices)
    old_to_new = {alive_list[j]: new_verts[j] for j in range(len(alive_list))}

    seen_edges: Set[Tuple[int, int]] = set()
    for i in alive_list:
        for nb in adj_sets[i] & alive:
            ek = (min(i, nb), max(i, nb))
            if ek not in seen_edges:
                seen_edges.add(ek)
                reduced.add_edge(Edge(old_to_new[i], old_to_new[nb]))

    # Initial colouring for the reduced graph
    init_colouring = {}
    for j, i in enumerate(alive_list):
        init_colouring[new_verts[j]] = colour[i]

    return reduced, aut_factor, init_colouring
```

Listing 7: *`preprocessing.py` — Preprocessing utilities*